



Experience Saudi As a Saudi

UMM AL-QURA UNIVERSITY
COLLEGE OF COMPUTING
SOFTWARE ENGINEERING DEPARTMENT

EXPERIENCE SAUDI AS A SAUDI

Ahmed Baksh
Ayman Omar
Yahya Alsharif
Khalid Wazna
Ahmed Felemban
Hussam Khan

Supervised by
Dr. Tariq A. Kashmeery

*Submitted for the partial fulfillment of the Bachelor of Software Engineering
degree*

College of Computer and Information Systems, Al-Abdiya Campus

June, 2026



ABSTRACT

Experience Saudi As a Saudi (ESAS) is a digital tourism platform designed to help visitors explore Saudi Arabia through authentic, locally curated experiences. Unlike traditional travel platforms that focus on generic attractions, ESAS provides travelers with Saudi-curated destinations and two travel modes: guided packages led by verified Saudi guides and self-guided digital roadmaps. The system includes secure booking workflows, provider verification, and backend-managed catalog and booking management into a single platform. ESAS aligns with Saudi Vision 2030 by promoting cultural immersion while supporting local small and medium-sized enterprises (SMEs). The platform is developed using Java 21, Spring Boot, PostgreSQL, and Flutter, with a backend-first architecture that ensures security, maintainability, and role-based access control. The current prototype demonstrates core traveler, provider, and administrator workflows with simulated payment flows, bilingual Arabic/English support, and a Docker-oriented local deployment setup. Real payment gateway integration, AI/weather recommendations, interactive maps, offline roadmaps, and production cloud deployment are planned for future work.



DECLARATION

We hereby declare that our dissertation is entirely our work and genuine/original. We understand that if any PLAGIARISM is discovered at any stage, our group will be assigned an F (FAIL) grade, and this may result in the withdrawal of our bachelor's degree.

Group members:

Name

Signature

Ahmed Baksh



Ayman Omar



Yahya Alsharif



Khalid Wazna



Ahmed Felemban



Hussam Khan





ACKNOWLEDGMENT

We would like to express our sincere gratitude to our project supervisor, Dr. Tariq A. Kashmeery, for his continuous guidance, encouragement, and valuable feedback throughout the development of this project. His expertise and support played a vital role in shaping our understanding and improving the quality of our work.

Finally, we would like to thank our families and friends for their constant support and encouragement throughout our academic journey. They were essential in helping us complete this work successfully.

TABLE OF CONTENTS

Chapter	Page
Chapter 1: Introduction	13
1.1. Introduction.....	13
1.2. Problem Domain	13
1.3. Problem Statement	13
1.4. Proposed System.....	13
1.4.1. Aims and Objectives	14
1.4.2. Proposed System Features	14
1.5. Project Methodology.....	15
1.6. Resource Requirement	15
1.6.1. Software (analysis/design/dev):	15
1.6.2. Hardware:.....	16
1.6.3. Minimum Client Requirements:	16
1.7. Report Layout	16
Chapter 2: Background / Existing Work	17
2.1. Introduction.....	17
2.2. Overview of Existing Projects	17
2.3. Limitations of existing projects	17
2.4. Innovations of our project (ESAS)	18
2.4.1. Comparative Overview	19
2.4.2. Market Enablers	20
Chapter 3: Software Requirements Specifications	21
3.1. Introduction.....	21

3.1.1.	Purpose.....	21
3.1.2.	Document Conventions.....	21
3.1.3.	Intended Audience and Reading Suggestions	21
3.1.4.	Product Scope	21
3.1.5.	References.....	22
3.2.	Overall Description.....	22
3.2.1.	Product Perspective.....	22
3.2.2.	Product Functions	22
3.2.3.	User Classes and Characteristics	24
3.2.4.	Operating Environment.....	26
3.2.5.	Design and Implementation Constraints.....	27
3.2.6.	User Documentation	28
3.2.7.	Assumptions and Dependencies	29
3.2.8.	BPMN Diagram	30
3.2.9.	BPMN Diagram Description.....	42
3.2.10.	Activity Diagram	46
3.2.11.	Activity Diagram Description.....	64
3.3.	External Interface Requirements.....	68
3.3.1.	User Interface.....	68
3.3.2.	Hardware Interface.....	68
3.3.3.	Software Interfaces	69
3.3.4.	Communication Interface.....	70
3.3.5.	Use Case Diagram.....	72
3.3.6.	Use Case Diagram Description.....	73
3.4.	System Features	73

3.4.1.	User Authentication and Role Management	73
3.4.2.	Public Catalog and Experience Browsing.....	74
3.4.3.	Experience Details and Filtering.....	75
3.4.4.	Traveler Cart	76
3.4.5.	Simulated Checkout and Booking	77
3.4.6.	Booking Management.....	77
3.4.7.	Wishlist and Wallet	78
3.4.8.	Provider Registration and Offering Management.....	79
3.4.9.	Admin Approval, Moderation, and Audit	79
3.4.10.	Reviews and Ratings.....	80
3.4.11.	Localization and User Interface	80
3.4.12.	Notifications and Communication	81
3.4.13.	Smart Roadmaps, Recommendations, and Maps.....	81
3.4.14.	Reports, Monitoring, and Audit	82
3.5.	Non-Functional Requirements	82
3.5.1.	Usability.....	82
3.5.2.	Reliability.....	82
3.5.3.	Security	83
3.6.	Performance Requirements.....	83
3.7.	Safety Requirements	83
3.8.	Security Requirements	83
3.9.	Software Quality Attributes	83
3.10.	Business Rules	84
3.11.	Other Requirements	84
Chapter 4: Software Design.....		85

4.1.	Design Overview	85
4.1.1.	Introduction.....	85
4.1.2.	Environment Overview	85
4.1.3.	Constraints and Assumptions.....	85
4.2.	User Interface Prototype	87
4.3.	Structural Design Overview.....	94
4.3.1.	Class Diagrams	94
4.3.2.	Class Diagram Description	97
4.3.3.	Sequence Diagrams.....	97
4.3.4.	Sequence Diagram Description.....	98
4.3.5.	ER Diagram	99
Chapter 5: System Implementation & Validation		101
5.1.	Introduction.....	101
5.2.	Implementation Environment	101
5.3.	System Architecture Implementation.....	102
5.4.	Backend Implementation	103
5.5.	Database Implementation.....	104
5.6.	Frontend Implementation.....	104
5.7.	Implemented Features	105
5.8.	Validation and Testing.....	106
5.8.1.	Backend Validation	107
5.8.2.	Frontend Validation.....	107
5.8.3.	Manual Test Scenarios	107
5.9.	Validation Results	108
5.10.	Implementation Limitations.....	109

Chapter 6: Conclusion & Future Work	110
6.1. Conclusion	110
6.2. Achievements.....	110
6.3. Limitations	111
6.4. Future Work	112
6.5. Final Statement	112
References / Bibliography.....	113

LIST OF FIGURES

Figure	Caption	Page
Figure 3-1	Sign Up BPMN Diagram	30
Figure 3-2	Login & Logout BPMN Diagram	31
Figure 3-3	View / Search and Select Experience BPMN Diagram	32
Figure 3-4	Traveler Profile BPMN Diagram	33
Figure 3-5	Generate Self-Guided Roadmap BPMN Diagram	34
Figure 3-6	Book Guided Package BPMN Diagram	35
Figure 3-7	Provider Manage Offerings BPMN Diagram	36
Figure 3-8	Provider Manage Schedule / Capacity BPMN Diagram	37
Figure 3-9	Contact Us BPMN Diagram	38
Figure 3-10	Admin Approve Providers BPMN Diagram	39
Figure 3-11	Admin Moderate Content BPMN Diagram	40
Figure 3-12	Communicate (Traveler ↔ Provider) BPMN Diagram	41
Figure 3-13	Sign Up Activity Diagram	46
Figure 3-14	Sign In Activity Diagram	47
Figure 3-15	Guest Progressive Onboarding Activity Diagram	48
Figure 3-16	Browse & Filter Activity Diagram	49
Figure 3-17	Map Explore Activity Diagram	50
Figure 3-18	Smart Roadmap Activity Diagram	51
Figure 3-19	Book Guided Package Activity Diagram	52
Figure 3-20	Payment Flow Activity Diagram	53
Figure 3-21	Limited Offer Activity Diagram	54
Figure 3-22	Provider Manage Offer Activity Diagram	55
Figure 3-23	Provider Availability & Pricing Activity Diagram	56
Figure 3-24	Admin Approve Provider Activity Diagram	57
Figure 3-25	Traveler Rate & Review Activity Diagram	58
Figure 3-26	Cancel & Refund Activity Diagram	59

Figure 3-27	Notifications Activity Diagram	60
Figure 3-28	Provider Campaign Lifecycle Activity Diagram	61
Figure 3-29	Ad Serving, Billing, & Anti-Abuse Activity Diagram	62
Figure 3-30	Password Reset Activity Diagram	63
Figure 3-31	Use Case Diagram	72
Figure 4-1	ESAS Demo Home Page	86
Figure 4-2	ESAS Demo Login Popup	87
Figure 4-3	ESAS Demo Guided Packages Preview Page	88
Figure 4-4	ESAS Demo Experience Search Page	89
Figure 4-5	ESAS Demo User Profile	90
Figure 4-6	ESAS Demo About Us Page Top Half	91
Figure 4-7	ESAS Demo About Us Page Bottom Half	92
Figure 4-8	Identity & Access Class Diagram	93
Figure 4-9	Experience & Booking Design Class Diagram	94
Figure 4-10	Governance & Marketing Class Diagram	95
Figure 4-11	Book Guided Experience Sequence Diagram	96
Figure 4-12	Generate Self-Guided Roadmap Sequence Diagram	97
Figure 4-13	Provider Creates/Updates Experience Sequence Diagram	97
Figure 4-14	ER Diagram and Relation Schema	98



LIST OF TABLES

Table	Caption	Page
Table 2-1	Comparing apps.	19
Table 2-2	Market enablers	20
Table 5-1	Implementation Environment	101
Table 5-2	Implemented Features Summary	105
Table 5-3	Manual Test Scenarios	107

Chapter 1: Introduction

1.1. Introduction

Saudi Arabia is rapidly emerging as a global destination under Vision 2030. While major landmarks are widely known, many visitors still struggle to access authentic, locally curated experiences that reflect Saudi culture, nature, and hospitality. ESAS (Experience Saudi As a Saudi) is a tourism platform that helps visitors live Saudi experiences “like a local” through curated recommendations, guided packages by Saudi guides, and self-guided roadmaps tailored to seasons or personal preferences, with integrated booking and backend-managed workflows.

1.2. Problem Domain

Tourism planning tools typically focus on generic attractions and fragmented information.

Travelers need:

Trusted local curation beyond the mainstream lists.

Season/weather-aware suggestions (e.g., hiking in winter, coastal activities in summer).

Easy access to licensed local guides and high-quality, ready-to-book packages.

Seamless end-to-end booking (transportation, activities, timing) and reliable ratings/reviews.

Options for self-guided roadmaps when a guide is not desired.

1.3. Problem Statement

Visitors to Saudi Arabia lack a single, reliable platform that: (1) discovers authentic, local experiences aligned with their preferences and the season; (2) connects them to verified Saudi guides with transparent quality signals; (3) offers complete tourism packages or self-guided plans with integrated payment and tracking; and (4) supports local businesses in a way that advances Vision 2030 goals.

1.4. Proposed System

ESAS is a web/mobile platform offering curated Saudi destinations, guided packages run by Saudi guides, and self-guided roadmaps. The system includes backend-managed booking workflows, provider verification, and a bilingual Arabic/English interface. The

current prototype focuses on backend-driven catalog browsing, simulated checkout, provider and admin workflows, and role-based access control.

1.4.1. Aims and Objectives

1. Deliver authentic cultural and adventure experiences designed/recommended by Saudis.
2. Personalize trips using preference + season/weather signals.
3. Provide ready-to-book guided packages and self-guided roadmaps.
4. Support local guides, vendors, and SMEs through fair visibility and bookings.
5. Ensure quality via ratings/reviews and safety/compliance checks.
6. Offer seamless booking, payment, and itinerary tracking flow.
7. Align with Vision 2030 by promoting cultural immersion and local economy impact.

1.4.2. Proposed System Features

The ESAS platform provides a complete digital ecosystem that connects travelers with authentic Saudi tourism experiences through the following features:

1. Saudi-Curated Destination Map

An interactive map featuring destinations selected by locals, organized by region and experience type (cultural, adventure, coastal, family, and religious).

2. Smart Experience Recommendations

Personalized suggestions based on season, weather, budget, and traveler preferences, with adaptive trip recommendations for different travel conditions.

3. Dual Travel Modes

- **Guided Packages:** Professionally organized tours led by verified Saudi guides.
- **Self-Guided Roadmaps:** Downloadable itineraries including navigation links, timing schedules, safety notes, and activity suggestions.

4. Unified Booking and Secure Payments

Integrated booking system supporting reservations, confirmations, and local payment gateways such as Mada, Apple Pay, and STC Pay.

5. Verified Guide and Provider System

Providers undergo approval before publishing offerings. Verified badges ensure trust and service quality.

6. Ratings and Reviews

After completing trips, travelers can submit reviews and ratings to improve transparency and service quality.

7. Local SME Promotion

Fair ranking system that highlights Saudi businesses and supports local entrepreneurs.

8. Offline Access Mode

Downloadable roadmaps and travel guides for travelers with limited connectivity.

9. Provider Management Dashboard

Tools for providers to manage offerings, schedules, pricing, and bookings.

10. Administration and Moderation Tools

Administrative dashboards for provider approval, content moderation, dispute management, and analytics.

11. Bilingual User Interface (Arabic & English)

The system supports both Arabic (RTL) and English (LTR) interfaces to enhance usability for domestic and international users.

12. Security and Privacy Controls

Authentication and authorization are implemented using Spring Security with JWT while ensuring compliance with Saudi data protection regulations.

1.5. Project Methodology

We follow a waterfall-style sequence to match academic reporting with iterative feedback where needed:

1) Requirements Analysis → 2) System Design (SRS, diagrams, data model) → 3) Implementation (frontend, backend) → 4) Testing (functional, usability, security) → 5) Deployment (local Docker prototype) → 6) Maintenance & Future Enhancements.

Deliverables per phase: SRS & UI mockups; BPMN/Activity/Use-case diagrams; ER/Class diagrams; Test plan & reports; Deployment guide.

1.6. Resource Requirement

1.6.1. Software (analysis/design/dev):

- **Documentation & diagrams:** Microsoft Word, Draw.io/Miro, Figma.

- **Frontend:** Flutter (Dart).
- **Maps & Weather:** Google Maps/Mapbox SDKs; reliable weather API.
- **Backend:** Spring Boot (Java 21).
- **Payments (KSA):** HyperPay / Tap / Moyasar (Mada, Apple Pay, STC Pay).
- **Database:** PostgreSQL with Flyway migrations.
- **Authentication:** Spring Security with JWT and BCrypt.
- **Notifications:** Firebase Cloud Messaging (FCM) may be considered for future notification support.
- **CI/CD & Hosting:** GitHub, Docker, Docker Compose.

1.6.2. Hardware:

- Developer laptop (≥ 16 GB RAM recommended).
- **Test devices:** Android (and iOS if applicable).
- Cloud server (2–4 vCPU, 8–16 GB RAM) for future staging/production deployment.

1.6.3. Minimum Client Requirements:

- Android 10+ (iOS 14+ if supported), stable internet.

1.7. Report Layout

This report is organized into six chapters. Chapter 1 introduces the ESAS project, including the problem domain, problem statement, proposed system, objectives, methodology, and resource requirements. Chapter 2 reviews background and existing work in tourism platforms and explains the limitations that ESAS addresses. Chapter 3 presents the software requirements specification, including product scope, user classes, interfaces, functional requirements, non-functional requirements, business rules, and system diagrams. Chapter 4 describes the software design, including the user interface prototype, system architecture, structural design, sequence diagrams, and database design. Chapter 5 explains the implementation and validation of the ESAS prototype, including the development environment, implemented modules, and testing activities. Chapter 6 concludes the report and presents limitations and future work.

Chapter 2: Background / Existing Work

2.1. Introduction

Saudi Arabia's tourism sector is expanding rapidly under Vision 2030. While major global platforms help travelers discover well-known attractions, visitors still struggle to access authentic, locally curated experiences and to coordinate end-to-end logistics across multiple cities. ESAS (Experience Saudi As a Saudi) addresses this gap with a Saudi-curated map, smart, season- and weather-aware suggestions, and two trip modes: guided packages with Saudi guides and self-guided roadmaps.

Compliance note (entry to the Kingdom): ESAS does not issue, process, or arrange visas. Travelers handle visas independently through official government channels. ESAS may show informational reminders and links only.

2.2. Overview of Existing Projects

Current planning tools used by visitors to Saudi Arabia include:

- **VisitSaudi (official gateway):** National content hub for destinations, events, and travel information.
- **TripAdvisor / Google Travel & Maps:** Global discovery, POIs, and review ecosystems.
- **GetYourGuide / Viator:** Marketplaces for bookable tours and activities run by third-party operators.
- **Airbnb Experiences / local DMCs:** Niche, guide-led or bespoke outings; quality varies by provider.

High-level observation: These platforms excel at breadth of supply and reviews, but provide limited Saudi-insider curation, weak seasonality/meteorology logic, and fragmented logistics for multi-city trips.

2.3. Limitations of existing projects

- **Generic recommendations:** Lists emphasize globally popular POIs rather than local favorites chosen by Saudis or a deeper cultural context.

- **Low season/weather awareness:** Few planners automatically adapt itineraries for heat, sandstorms, or winter hiking windows (e.g., Asir/Tabuk).
- **Fragmented logistics:** Tickets, transport, permits/gear, timing, and payments are split across apps—raising booking drop-off.
- **Guide verification:** Hard to identify licensed Saudi guides with transparent track records and immersive formats.
- **Shallow self-guided planning:** Many platforms sell tours; fewer generate rich roadmaps (timed stops, navigation links, offline packs, packing/safety lists).
- **Local SME visibility:** Small Saudi vendors (heritage cafés, craft studios, desert outfitters) are under-indexed compared to large operators.

2.4. Innovations of our project (ESAS)

ESAS introduces a Saudi-first, context-aware approach to solve the gaps above:

1. Saudi-Curated Interactive Map

Destinations and micro-experiences hand-picked by Saudis, organized by city/region and theme (spiritual, cultural, adventure, coastal, family).

2. Smart Experience Suggestions (AI-assisted or manual)

Itineraries adapt to season + forecast/weather, trip length, budget, and traveler style, with automatic re-planning when conditions change.

3. Two Trip Modes

(a) Guided Packages with Saudi Guides: end-to-end bundles covering transport, timing, tickets/permits, gear when applicable.

(b) Self-Guided Roadmaps (no guide): timed plans with navigation links, optional offline map packs, packing & safety checklists.

4. Unified Booking & Local Payments

Single checkout, reservation tracking, notifications; support for popular local payment methods (e.g., Mada, Apple Pay, STC Pay via certified gateways).

5. Quality Signals & Safety

Verified-guide badges, post-trip ratings & reviews, clear SLAs, and issue-resolution.

6. Local SME Enablement

Dedicated storefronts and a fair ranking to promote small, authentic Saudi providers.

7. Regulatory & Entry Compliance (Informational Only)

ESAS does not handle visas or any immigration services. The app may show FYI links to official government sites when users ask about entry requirements.

2.4.1. Comparative Overview

Criterion	VisitSaudi (Official)	TripAdvisor/ Google	GetYourGuide/ Viator	ESAS (Proposed)
Saudi-specific curation	Good (broad)	Broad, global	Good supply	High (local- first)
Season & weather awareness	Limited	Limited	Limited	Adaptive (AI/manual)
Saudi licensed guides	Partial	Rarely emphasized	Varies by supplier	Core focus
Self-guided roadmaps	Limited	Generic	Limited	Detailed & navigable
Unified booking & local payments	Partial	External	Good	Unified + local rails
SME visibility	Partial	Low	Low-medium	Promoted by design

Table 2-1: Comparing apps.

2.4.2. Market Enablers

Enabler	What it means for tourists	ESAS stance
Rapid tourism growth under Vision 2030	Rising interest in authentic, bookable experiences	ESAS supplies curated content & bookings
Official entry/visa programs (handled by the government only)	Easier planning for eligible tourists	ESAS does not issue/facilitate visas; it shows official info links only
Digital payments adoption in KSA	Frictionless checkout expectations	Integrate certified local gateways in future work
Domestic transport & inter-city connectivity	Multi-city itineraries are common	Cross-city catalog & roadmap logic

Table 2-2: Market enablers (context, not services provided by ESAS).

Chapter 3: Software Requirements Specifications

3.1. Introduction

3.1.1. Purpose

This chapter outlines the software requirements and specifications. It also provides detailed information about the platform's additional features.

3.1.2. Document Conventions

The GP-Template-UQU-CIS-SE template was adopted to write the SRS chapter with some notable changes. Firstly, the text uses a black “Times New Roman” font and is justified. Section headers are centered, bold, and size 18. Headers and sub-headers are also bold, size 14. The body is size 12. Table and figure names are blue, italicized, and size 10. Bold words are titles, headers, sub-headers, list numbers, and important words or sentences; words with “Quotation marks” are literal meanings or names. Finally, document sections are ordered by numbers and indentation, from section headers to sub-headers.

3.1.3. Intended Audience and Reading Suggestions

This document is intended for developers, designers, project supervisors, testers, and stakeholders involved in the ESAS platform. It is also suitable for academic evaluators reviewing the project’s requirements and design. Readers are encouraged to start with the overall description (Section 3.2) and product perspective, then proceed to user classes, functional requirements, and interface descriptions.

3.1.4. Product Scope

ESAS is a tourism platform designed to deliver authentic, locally curated Saudi experiences to visitors. The product's core purpose is to simplify and enhance trip planning by providing a smart interactive map of Saudi destinations, AI-assisted experience suggestions that adapt to season and weather, and two main service modes: guided packages led by verified Saudi guides and self-guided roadmaps. The platform will include integrated booking, secure local payment processing, and a review system. It is important to note that the product's scope explicitly excludes visa issuance or immigration services, which remain the sole responsibility of official government channels.

3.1.5. References

This SRS refers to the following documents and resources:

1. ESAS project documentation and implementation roadmap.
2. ESAS OpenAPI specification, used as the API contract reference.
3. ESAS system status model, used as the reference for status values and transitions where applicable.
4. ESAS user interface design specifications and prototype screens.
5. Spring Boot and Spring Security documentation for backend and authentication design.
6. PostgreSQL and Flyway documentation for database and migration design.
7. Flutter documentation for frontend and localization implementation.
8. Docker documentation for local deployment and container-based database setup.
9. Saudi tourism and Vision 2030 public resources used for background context.
10. Official government resources for visa/entry information, used only as informational references because ESAS does not issue or process visas.

If a final bibliography is added later, these references should be consistent with it.

3.2. Overall Description

3.2.1. Product Perspective

ESAS is conceived as a comprehensive tourism platform designed to bridge the gap between international visitors and authentic Saudi experiences. It provides a centralized solution for discovering and booking culturally immersive activities and multi-city itineraries, curated by locals and tailored to seasonal and weather conditions. The platform empowers tourists by simplifying end-to-end trip planning with integrated booking and payment, while simultaneously supporting the local economy by increasing the visibility and revenue of Saudi guides and small to medium-sized tourism enterprises, thereby aligning with the national objectives of Vision 2030.

3.2.2. Product Functions

The ESAS platform provides a range of functional services categorized by user type to ensure usability, scalability, and security.

1. Traveler (Visitor) Functions

- Register and log in to an account.
- Browse destinations via map or list view.
- Filter experiences by location, category, season, and budget.
- Generate personalized self-guided roadmaps.
- Book guided packages.
- View booking history and upcoming itineraries.
- Make payments through simulated payment flows in the current prototype, with approved real gateways planned for future work.
- Submit ratings and reviews (planned/future work).
- Download offline guides and trip plans (planned/future work).
- Contact providers directly through the system (planned/future work).

2. Saudi Guide / Provider Functions

- Create a provider account with verification.
- Add, update, and remove tour listings.
- Upload photos, prices, descriptions, and schedules.
- Manage booking requests and availability.
- Set participant limits and timelines.
- Respond to traveler inquiries and reviews.
- Request payouts and view transaction records (planned/future work).
- Launch limited-time promotional offers (planned/future work).
- Manage advertising campaigns (planned/future work).

3. Administrator Functions

- Approve or reject provider registrations.
- Moderate experiences and user reviews.
- Manage disputes and compliance violations.
- Monitor system activities and logs.
- Enforce business rules and abuse control.
- Generate administrative reports.

- Assign role-based access privileges.
- Control content quality and ranking policies.

4. System Support Functions

- User authentication using JWT.
- Notification dispatch through email, push, or SMS (planned/future work).
- Real payment gateway validation (planned/future work).
- Analytics and performance monitoring.
- Error handling and logging.
- Backup and recovery services.
- Data encryption and protection.

3.2.3. User Classes and Characteristics

ESAS is a single platform serving three primary user classes and several supporting classes. Classes differ by frequency of use, the subset of functions they support, technical expertise, and privilege (RBAC).

3.2.3.1. Primary User Classes

1. Tourist (Visitor)

- **Typical goals:** Discover curated experiences; generate self-guided roadmaps; book guided packages; pay; rate/review; (optional) download offline pack.
- **Frequency:** Occasional to moderate (trip planning windows, on-trip usage).
- **Expertise:** Low (consumer).
- **Privilege:** Lowest; access to own data and public content only.
- **Distinct needs:** Clear AR/EN content, simple flows, secure checkout, and offline access in poor-coverage areas.

2. Saudi Guide / Experience Provider (SME)

- **Typical goals:** Onboarding & verification (KYC); publish/manage offerings; set schedules/capacity/pricing; view bookings; handle cancellations/refunds per policy; respond to reviews.
- **Frequency:** Moderate to high (daily/weekly operations).

- **Expertise:** Medium (business user).
- **Privilege:** Manage own offerings/bookings; no access to other providers' data.
- **Distinct needs:** Reliable calendar/capacity tools, payout status visibility, and policy templates.

3. Administrator (Platform Admin)

- **Typical goals:** Content curation; provider verification; quality & SLA oversight; dispute handling; analytics & reporting; RBAC & audit.
- **Frequency:** High (daily).
- **Expertise:** Medium–High (ops).
- **Privilege:** Highest (scoped by admin roles: content, trust & safety, ops).
- **Distinct needs:** Full audit trails, impersonation (view-only) for support, and safeguard rails on destructive actions.

3.2.3.2. Supporting classes

1. Support/Moderator

- **Goals:** Handle tickets/complaints; moderate reviews/content; assist travelers/providers.
- **Frequency:** High. **Expertise,** Medium. **Privilege:** Elevated but scoped (no schema changes).
- **Distinct needs:** Case management, canned replies, SLA timers, and redaction tools.

2. Finance & Compliance Ops

- **Goals:** Refunds/chargebacks; settlement & payouts; invoice/receipts; compliance checks.
- **Frequency:** Moderate. **Expertise,** Medium–High **Privilege:** Financial data scoped to necessity.
- **Distinct needs:** Escrow/settlement views, reconciliation exports, and audit logs.

Note: Traveler and Guide/Provider drive marketplace value and are the highest priority; Admin/Support/Finance are critical for governance and reliability.

3.2.4. Operating Environment

The following environment describes both the current prototype environment and the target production environment. Items such as caching, cloud object storage, centralized monitoring, alerts, and strict availability targets are planned for production deployment and are not fully implemented in the current prototype.

1. Runtime environment

- **Operating System:** Linux (Ubuntu 22.04) or Windows.
- **Platform type:** Spring Boot standalone (Tomcat) as backend, Flutter frontend.
- **Time/Locale:** Asia/Riyadh (UTC+3), default currency SAR, bilingual support (Arabic / English).
- **Deployment model:** Docker Compose (local PostgreSQL/demo environment). Authentication handled by Spring Security + JWT.

Core services

- REST API served through Spring Web MVC with HTTPS enabled
- Authentication & authorization through Spring Security with JWT.

Data layer

- **Relational database:** PostgreSQL (implemented with Flyway migrations).
- **ORM:** Spring Data JPA with Hibernate.
- **Caching:** Redis or Redisson.
- Offline storage (downloadable roadmaps) stored via object storage (Azure Blob, AWS S3, or MinIO).

Network and browser support

- HTTPS (TLS 1.2+).
- **Browser support:** Chrome, Edge, Firefox, Safari on desktop and mobile.
- For map functions, the browser geolocation API is required.

Monitoring and logs

- Application metrics exposed via Spring Boot Actuator.

- Logging implemented via SLF4J + Logback, aggregated with ELK stack or cloud monitoring.
- Alerts generated from Prometheus, Grafana, or cloud-native equivalents.

Acceptance criteria

- All services run successfully on Linux or Windows using Spring Boot.
- All REST endpoints reachable over HTTPS.
- p95 API latency ≤ 800 ms.
- Minimum 99.5% availability.

3.2.5. Design and Implementation Constraints

1. Platform baseline

- The system is implemented using Java 21+ with Spring Boot.

Security

- All communication is encrypted via HTTPS/TLS.
- Authentication and authorization implemented using Spring Security + JWT tokens.
- Passwords stored using BCrypt hashing.
- No cardholder data should be stored by ESAS. The current prototype uses simulated payment flows, while future production payment processing should be handled by Saudi-approved payment gateways.

Data & Storage

- **Database:** PostgreSQL 14+ (primary).
- **ORM:** Spring Data JPA (Hibernate) for persistence.
- Media stored securely in cloud object storage.

APIs & Architecture

- RESTful APIs built using Spring Web MVC with JSON payloads.
- API versioning implemented through URI versioning or headers.
- Optional modularization supported through Spring Boot multi-module structure.

Logging & backup

- Logging framework: SLF4J + Logback.
- Weekly or daily database backups in SQL dump format.
- Centralized logging through ELK stack or cloud-based logging.

Localization & Locale

- Full Arabic (RTL) and English (LTR) support.

Scope Boundaries

- Out of scope: visa/immigration processing.
- Providers can only access their own offerings and bookings.

Acceptance

- Build using Maven or Gradle.
- Must deploy successfully as a Docker container.

3.2.6. User Documentation

The system will include a comprehensive **user manual** explaining how to use the main features of the platform.

This documentation will be provided in PDF format and written in Arabic and English.

It will include:

- Overview of the system and its purpose.
- Login instructions for each user role (Traveler, Provider, Admin).
- Step-by-step guide for booking, managing offers, and viewing reports.
- Common issues and troubleshooting tips (e.g., login errors, missing data).
- Contact information for technical support or feedback.

The manual will be updated after testing to reflect the final interface and the features included in the production release.

3.2.7. Assumptions and Dependencies

3.2.7.1. Assumptions

- The platform will be accessed primarily through modern web browsers on desktop and mobile devices.
- All users (Traveler, Provider, Admin) are expected to have basic digital literacy to use the platform effectively.
- The system operates with real data related to users, bookings, and payments, following all necessary privacy and data protection standards.
- The payment module simulates transactions for demonstration purposes but is designed to support future integration with verified payment gateways.
- The system will be monitored and updated periodically by the development team to maintain stability and security.

3.2.7.2. Dependencies

- The system depends on a relational database (PostgreSQL 14+) for managing persistent data.
- The system depends on the Java Runtime Environment (JRE 21+) and Spring Boot for backend execution.
- Email and notification services are planned future dependencies for user communication.
- A stable internet connection is required for system access and synchronization.
- Optional external APIs (such as maps or currency converters) may be integrated later to enhance usability.

3.2.8. BPMN Diagram

3.2.8.1. Sign Up

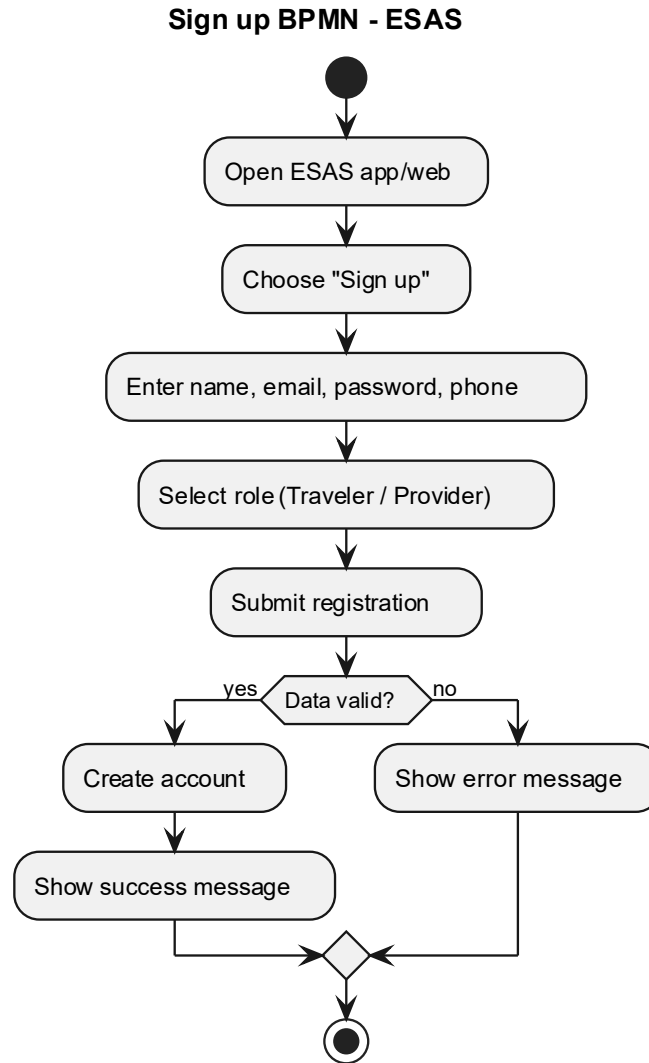


Figure 3-1: Sign Up BPMN Diagram

3.2.8.2. Login & Logout

Login & Logout BPMN - ESAS

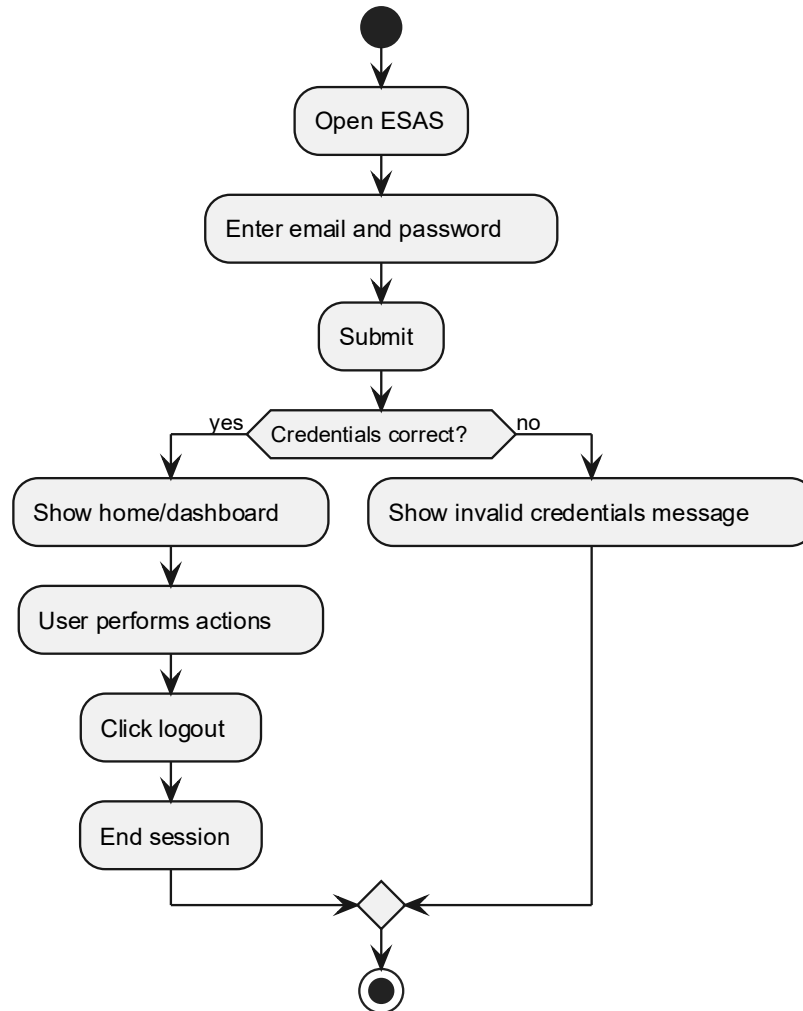


Figure 3-2: Login & Logout BPMN Diagram



3.2.8.3. View / Search and Select Experience

View/Search and Select Experience BPMN - ESAS

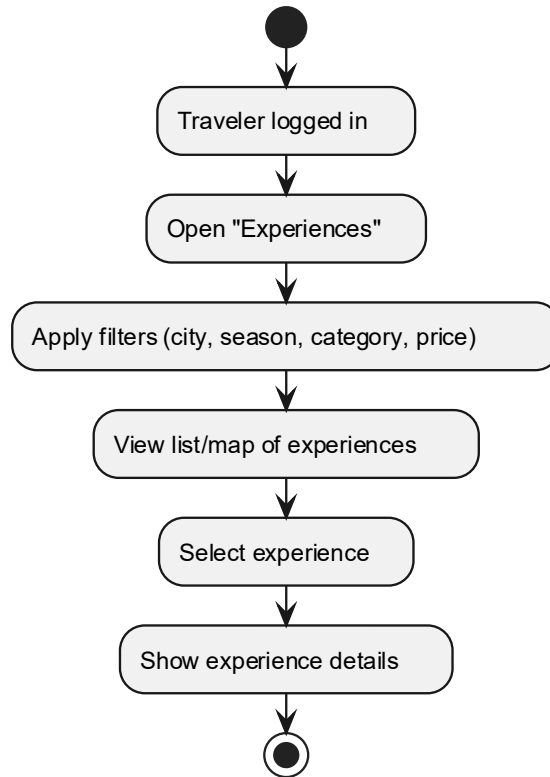


Figure 3-3: View / Search and Select Experience BPMN Diagram

3.2.8.4. Traveler Profile

Traveler Profile BPMN - ESAS

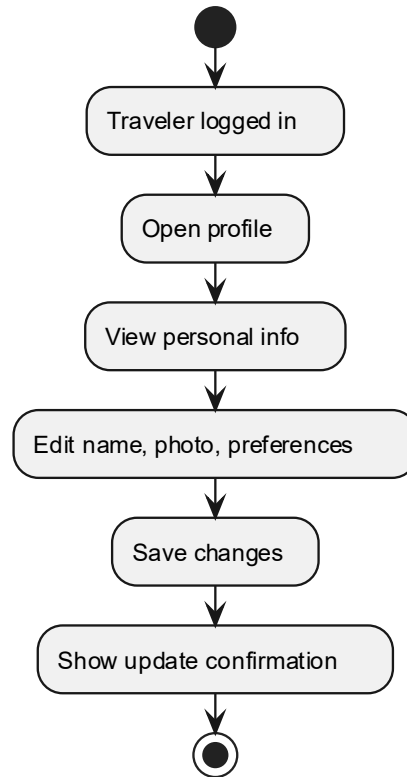


Figure 3-4: Traveler Profile BPMN Diagram

3.2.8.5. Generate Self-Guided Roadmap

Generate Self-Guided Roadmap BPMN - ESAS

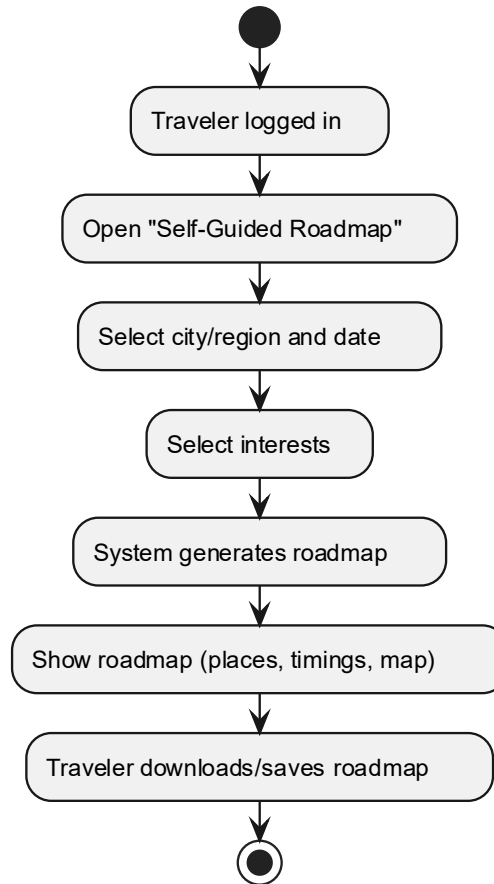


Figure 3-5: Generate Self-Guided Roadmap BPMN Diagram

3.2.8.6. Book Guided Package

Book Guided Package BPMN - ESAS

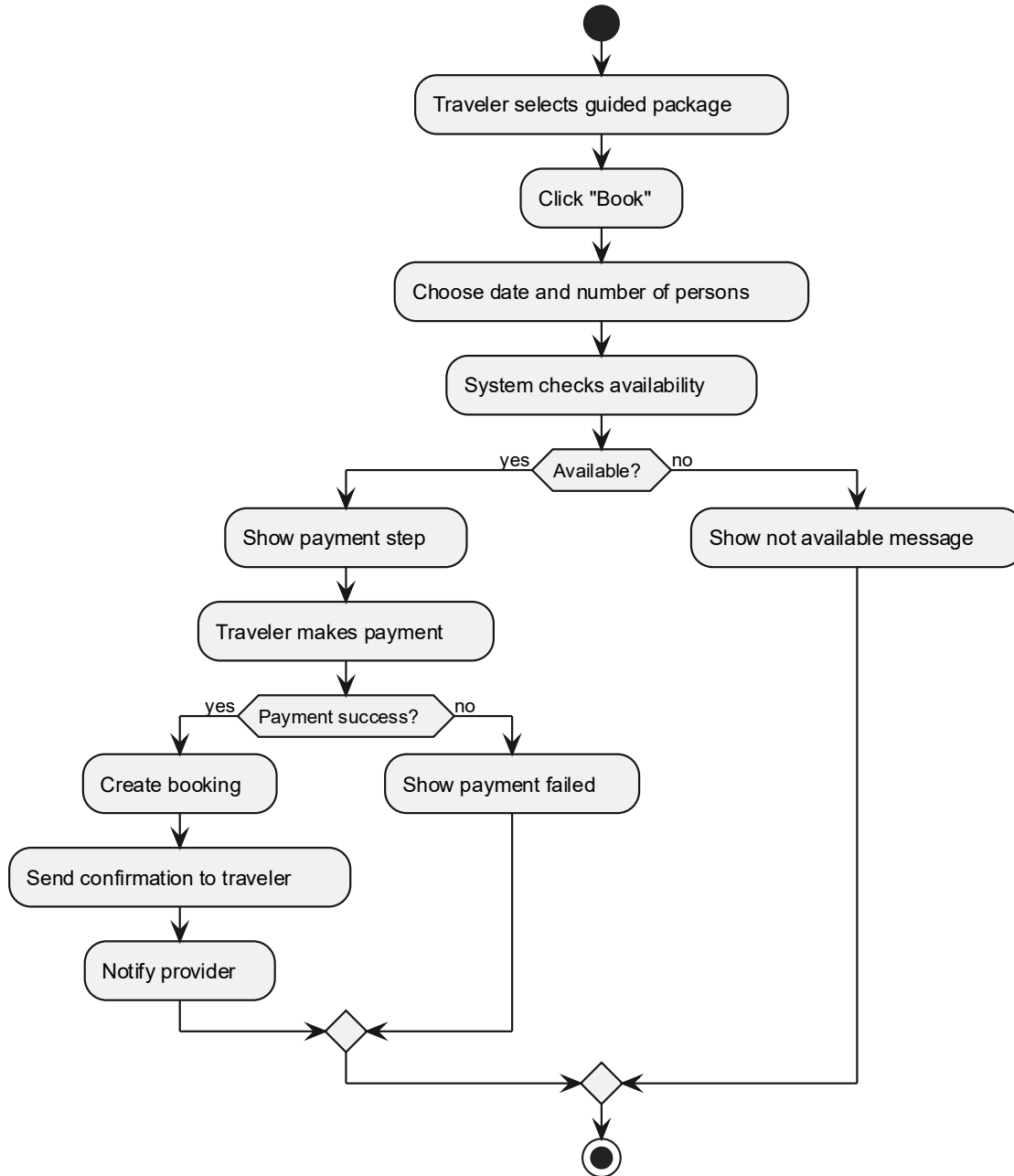


Figure 3-6: Book Guided Package BPMN Diagram

3.2.8.7. Provider Manage Offerings

Provider Manage Offerings BPMN - ESAS

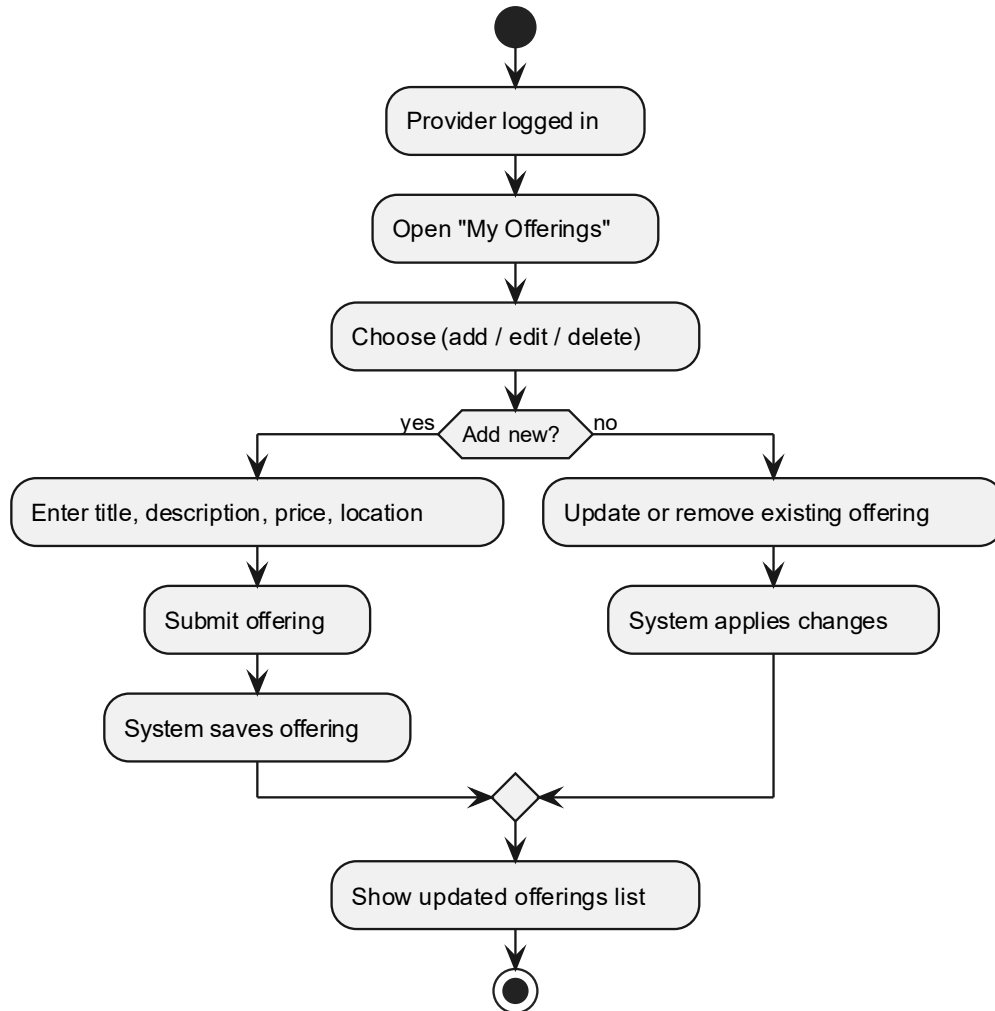


Figure 3-7: Provider Manage Offerings BPMN Diagram

3.2.8.8. Provider Manage Schedule / Capacity

Provider Manage Schedule BPMN - ESAS

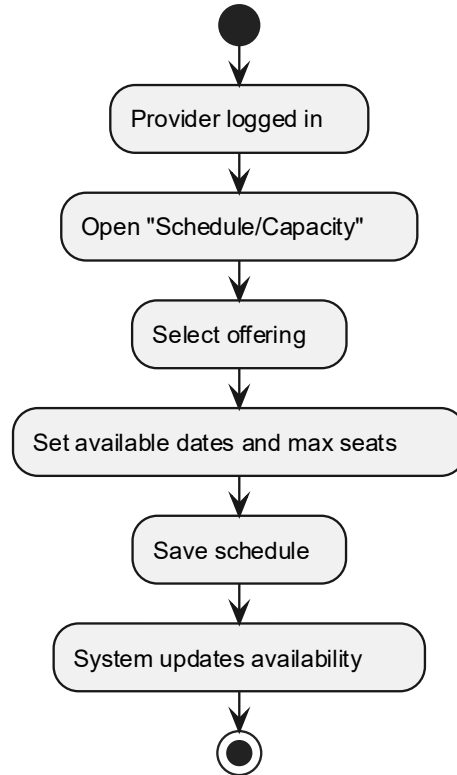


Figure 3-8: Provider Manage Schedule / Capacity BPMN Diagram

3.2.8.9. Contact Us

Contact Us BPMN - ESAS

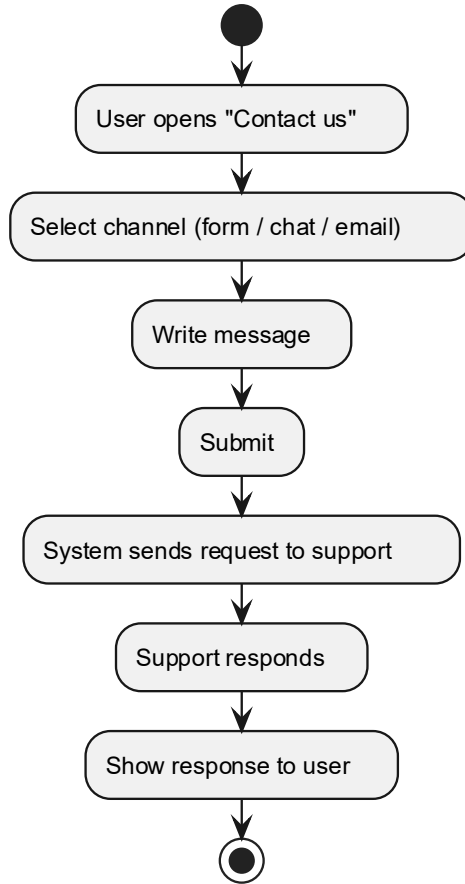


Figure 3-9: Contact Us BPMN Diagram

3.2.8.10. Admin Approve Providers

Admin Approve Providers BPMN - ESAS

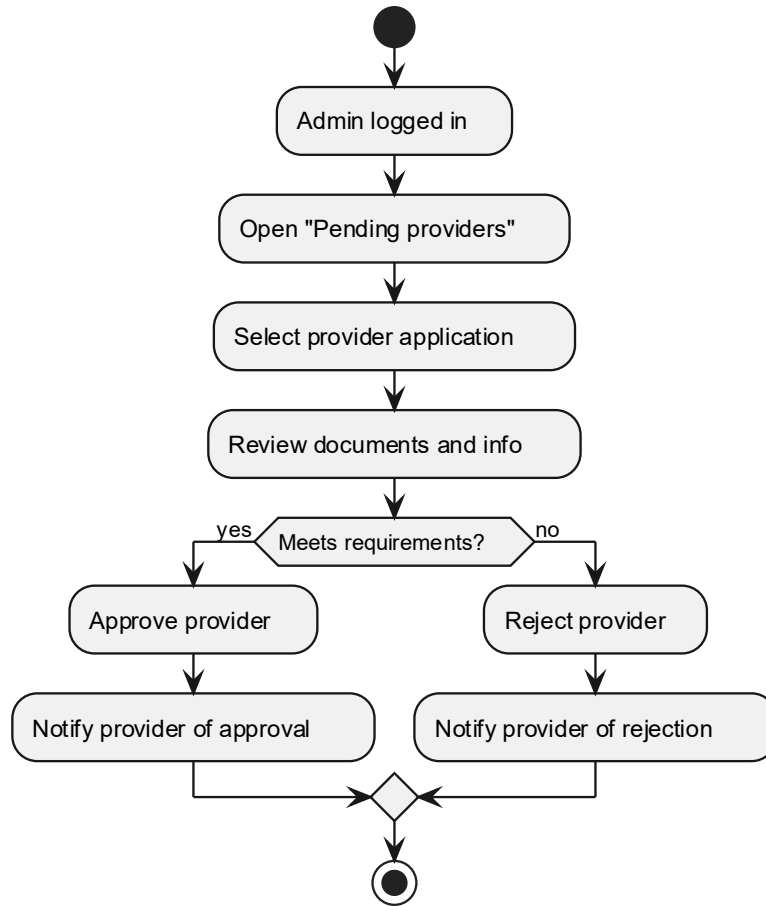


Figure 3-10: Admin Approve Providers BPMN Diagram

3.2.8.11. Admin Moderate Content

Admin Moderate Content BPMN - ESAS

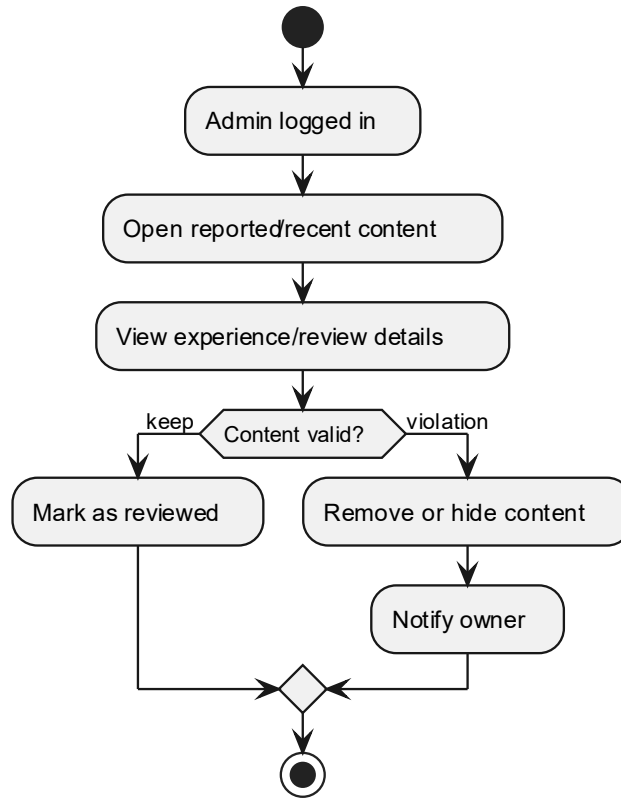


Figure 3-11: Admin Moderate Content BPMN Diagram

3.2.8.12. Communicate (Traveler ↔ Provider)

Communicate BPMN - ESAS

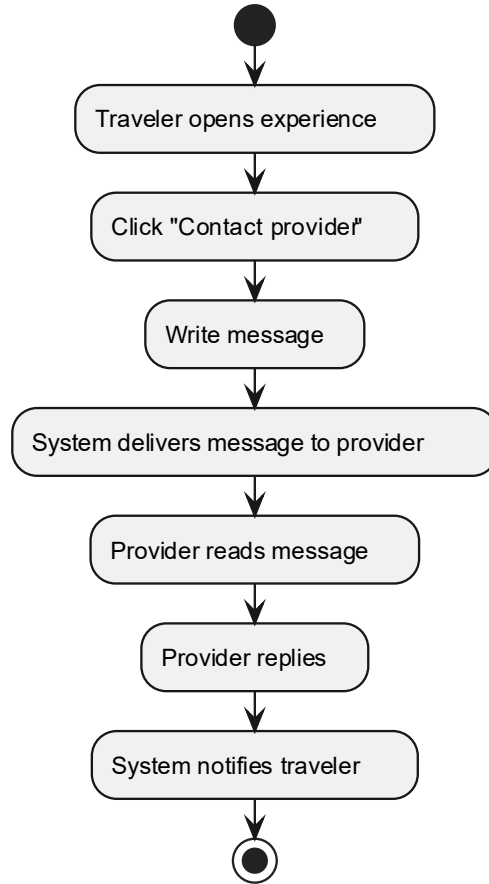


Figure 3-12: Communicate (Traveler ↔ Provider) BPMN Diagram

3.2.9. BPMN Diagram Description

Editorial Note: Each BPMN description below clarifies the responsibility boundaries between user actions and system-automated actions, highlights key decision points, and explains state transitions for critical entities. Cross-references to related activity diagrams are provided where applicable to avoid redundancy while preserving process clarity.

3.2.9.1. Sign up

This process represents how a new user registers on the ESAS platform. The user opens the application or website, chooses to sign up, fills in required information (name, email, password, phone), selects their role as a traveler or provider, and submits the registration form. The system validates the data, and either creates a new account or returns an error message if the information is invalid.

Enhancement Note: User actions and system validations are logically separated, where the traveler provides registration data and the system performs verification, account creation, and OTP validation. In case of validation failure, the process safely returns the user to the data entry step with clear feedback. Upon successful verification, the user account state transitions from pending to active.

3.2.9.2. Login & Logout

This process describes how a user accesses and leaves the ESAS system. The user enters their email and password to log in. If the credentials are correct, the user gains access to the dashboard or homepage. After performing actions within the system, the user can log out to end the session securely.

Enhancement Note: The authentication process separates user credential submission from system-side verification and session handling. Successful login results in secure sessions or token issuance, while logout explicitly invalidates active sessions to prevent unauthorized reuse.

3.2.9.3. View / Search and Select Experience

This process explains how travelers explore experiences. The traveler opens the “Experiences” section, applies filters (city, season, category, price), views available

experiences on a list or interactive map, and selects one to see detailed information about it.

Enhancement Note: This process focuses on discovery and selection rather than booking. Filtering and ranking are handled automatically by the system based on user-selected criteria and contextual signals, while this BPMN complements the map exploration process without duplicating its logic.

3.2.9.4. Traveler Profile

This diagram shows how travelers manage their personal profiles. The traveler opens the profile page, views or edits their personal details, updates information like name, picture, or preferences, and saves the changes. The system confirms the update successfully.

Enhancement Note: Profile management clearly distinguishes between traveler-initiated updates and system-side validation and persistence. All profile changes are validated and securely stored to ensure data accuracy and privacy.

3.2.9.5. Generate Self-Guided Roadmap

This process illustrates how travelers generate a personalized self-guided roadmap. The traveler opens the roadmap feature, selects a region and travel date, chooses preferred categories or interests, and the system automatically generates a trip plan with locations, timings, and maps that can be saved or downloaded.

Enhancement Note: The roadmap generation process combines traveler preferences with system intelligence, including time, location, and contextual factors. The system automatically assembles a structured itinerary that can be saved or accessed offline, ensuring usability even in limited connectivity scenarios.

3.2.9.6. Book Guided Package

This diagram represents the booking flow for guided packages. The traveler selects a guided experience, specifies the date and participants, and submits the booking. The system checks availability and proceeds with payment. After successful payment, the system creates a booking record, sends confirmation to the traveler, and notifies the provider. If payment or availability fails, the system shows an error message.

Enhancement Note: Booking confirmation is strictly dependent on successful availability checks and payment completion. The process ensures transactional integrity by confirming the booking only after payment succeeds, while failures result in safe rollback without partial reservations.

3.2.9.7. Provider Manage Offerings

This process shows how a provider manages their offerings. After logging in, the provider navigates to “My Offerings” and can add, update, or delete experiences. When adding new offerings, the provider inputs the title, description, price, and location. The system saves or updates the information accordingly.

Enhancement Note: Provider actions for creating or modifying offerings are clearly separated from system validation and publishing controls. Newly created or updated offerings may require administrative approval before becoming publicly visible, ensuring content consistency and compliance.

3.2.9.8. Provider Manage Schedule / Capacity

This process explains how providers manage availability. The provider opens the scheduling section, selects an offer, sets available dates and the maximum number of participants, and saves the data. The system updates availability information to reflect these changes.

Enhancement Note: The system enforces availability and capacity rules to prevent conflicts or overbooking. Schedule updates are validated against existing confirmed bookings to maintain operational accuracy.

3.2.9.9. Contact Us

This process represents how users contact support. A user opens the “Contact Us” page, selects a channel (form, chat, or email), writes a message, and submits it. The system forwards the message to the support team, which then replies to the user’s inquiry.

Enhancement Note: All contact requests are captured and routed through a centralized support workflow. The system ensures that user inquiries are delivered to the appropriate support channel and that responses are tracked for accountability.

3.2.9.10. Admin Approve Providers

This process describes how the administrator reviews and approves provider registrations. The admin opens the list of pending applications, checks submitted documents, and determines eligibility. If the requirements are met, the provider is approved and notified; otherwise, the admin rejects the application and sends a rejection message.

Enhancement Note: Provider approval decisions are based on predefined verification criteria, including submitted documentation and platform policies. Each approval or rejection is recorded to ensure transparency and auditability.

3.2.9.11. Admin Moderate Content

This process shows how administrators moderate content within the platform. The admin reviews reported or newly added experiences and user reviews. If content violates platform rules, it is removed or hidden, and the owner is notified; otherwise, the content is marked as reviewed and kept active.

Enhancement Note: Content moderation actions are governed by platform rules and community guidelines. The system allows administrators to review, remove, or retain content while notifying relevant users of moderation outcomes.

3.2.9.12. Communicate (Traveler ↔ Provider)

This process illustrates the communication between travelers and providers. The traveler opens the experience page and sends a message to the provider through the system. The provider receives and replies to the message, and the traveler is notified about the response, ensuring two-way communication within the ESAS platform.

Enhancement Note: Communication between travelers and providers is handled asynchronously through the platform. Messages are delivered with notification support, ensuring timely responses while maintaining user privacy and controlled interaction.

3.2.10. Activity Diagram

3.2.10.1. Sign up

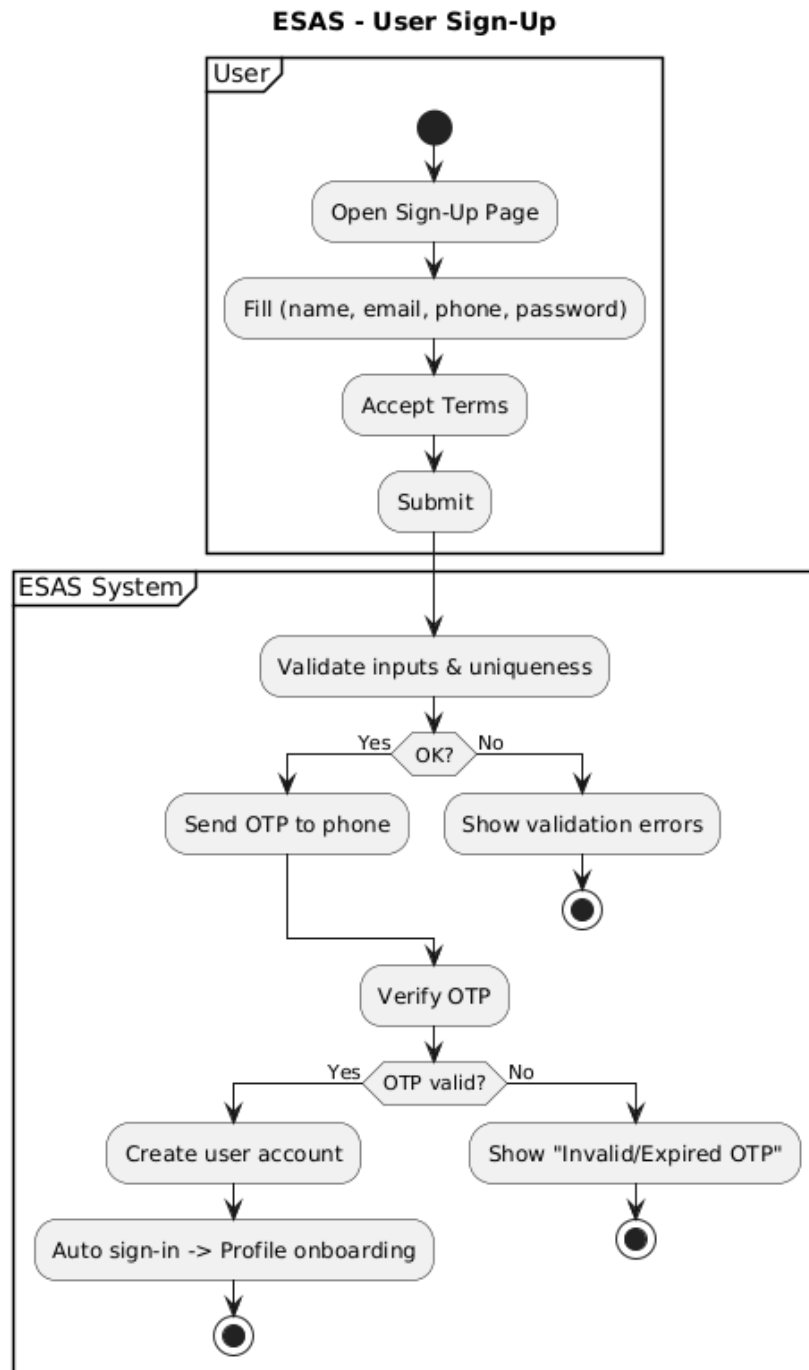


Figure 3-13: Sign Up Activity Diagram

3.2.10.2. Sign in

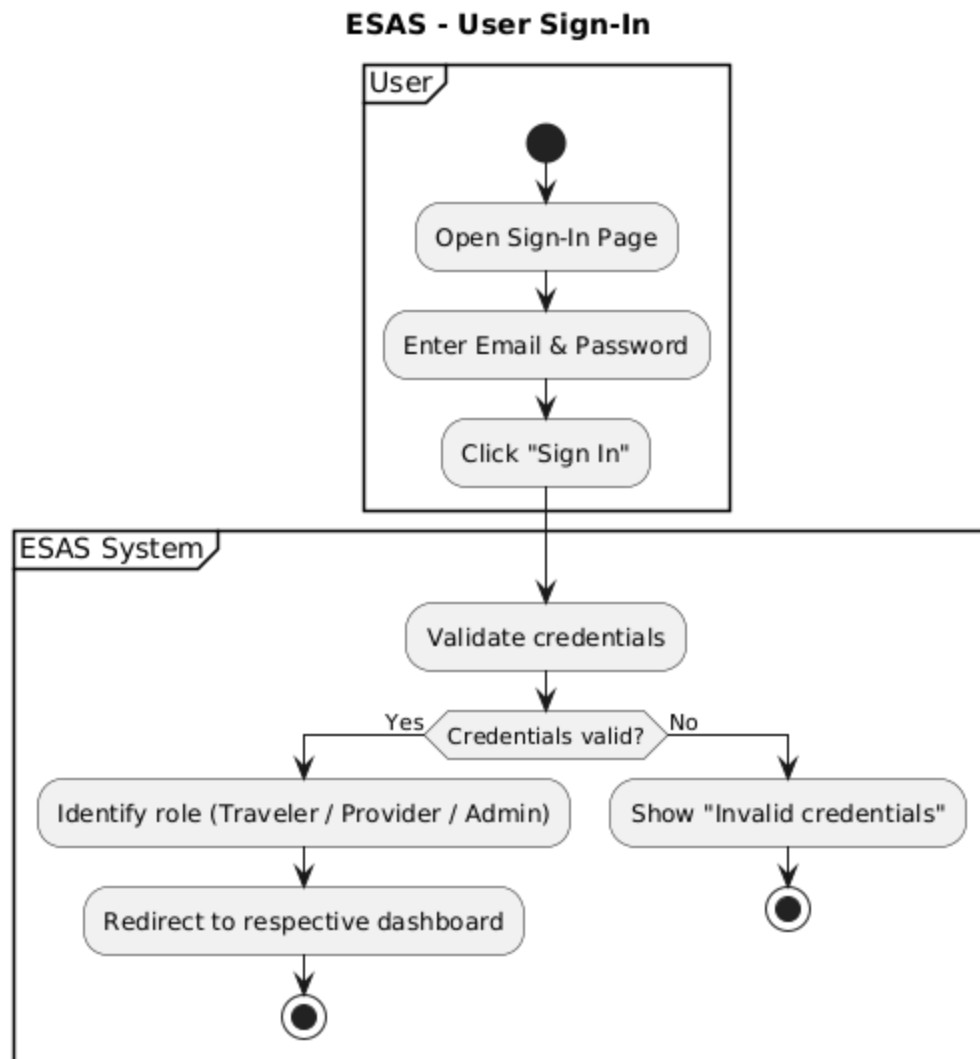


Figure 3-14: Sign In Activity Diagram

3.2.10.3. Guess Progressive Onboarding

ESAS - Browse First, Then Auth (Progressive Onboarding)

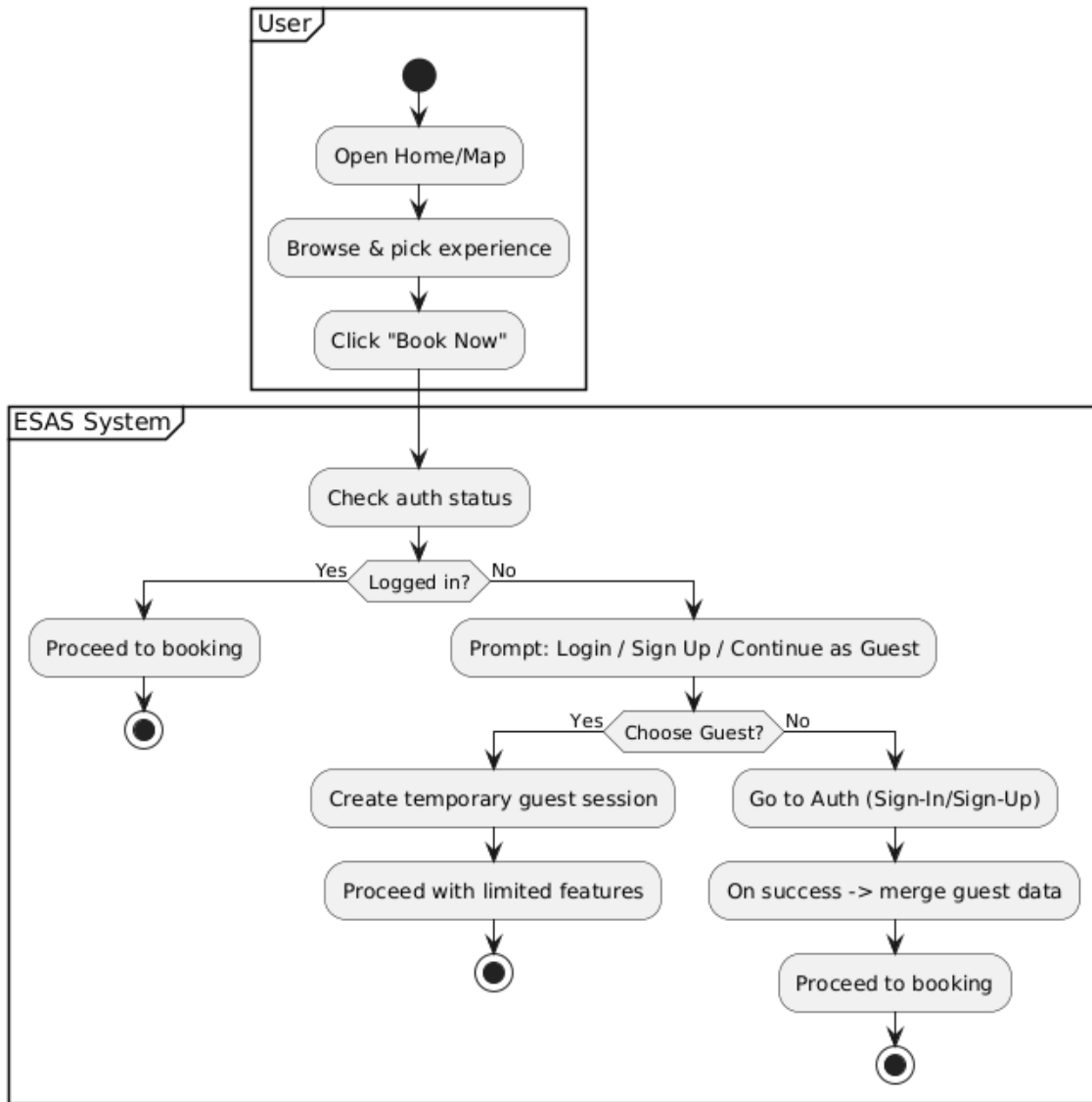


Figure 3-15: Guest Progressive Onboarding Activity Diagram

3.2.10.4. Browse & Filter

ESAS - Browse & Filter (Discovery Only)

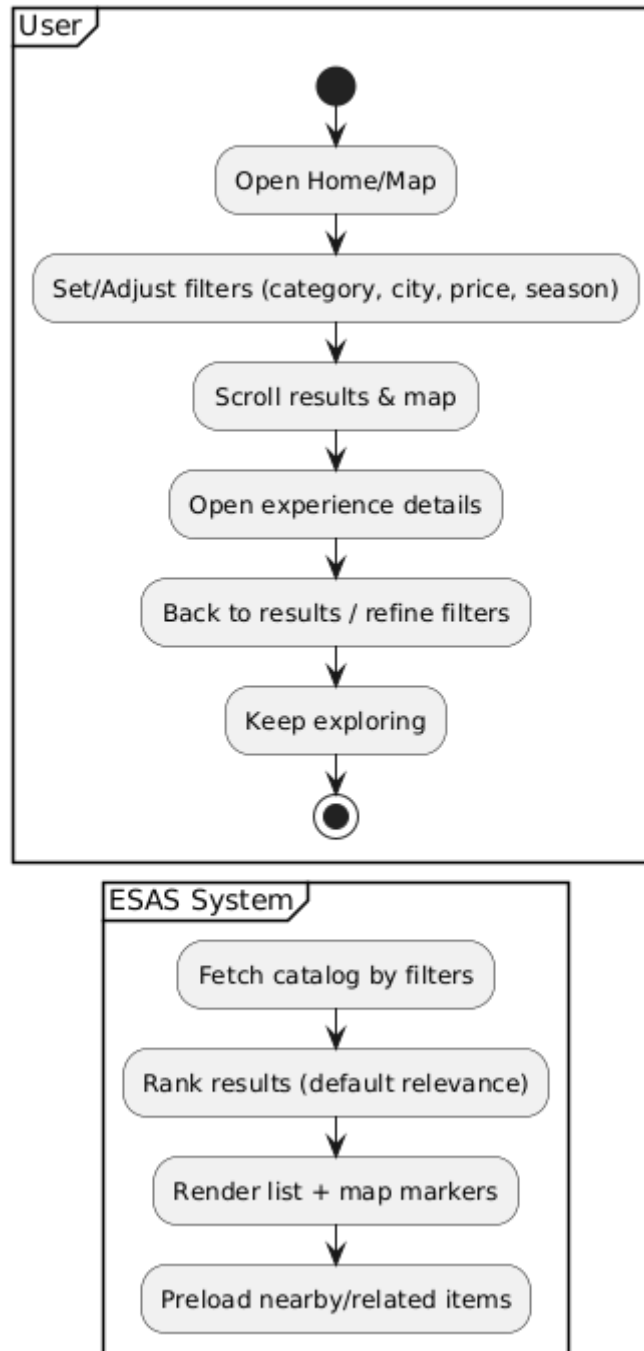


Figure 3-16: Browse & Filter Activity Diagram

3.2.10.5. Map Explore

ESAS - Map Explore: Organic + Sponsored Pins (Blending, Capping, Targeting)

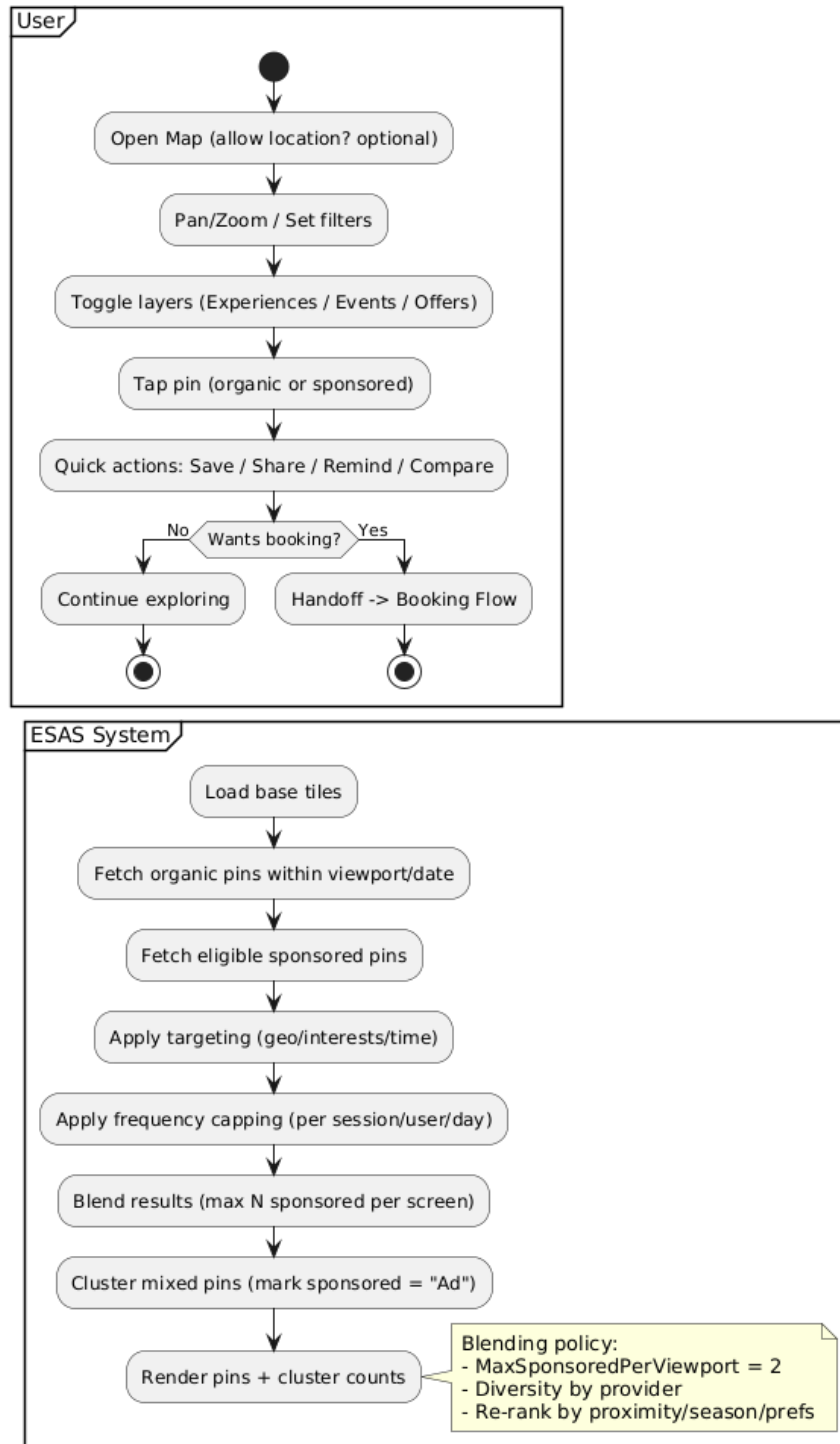


Figure 3-17: Map Explore Activity Diagram

3.2.10.6. Smart Roadmap

ESAS - Smart Roadmap with Events & Sponsored Pins

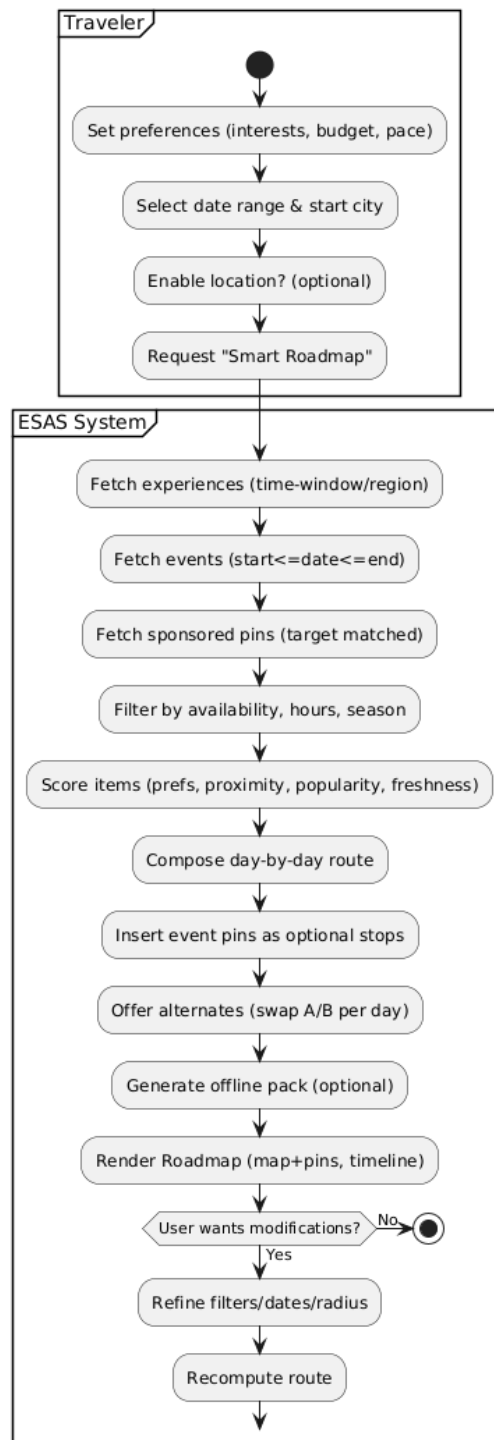


Figure 3-18: Smart Roadmap Activity Diagram

3.2.10.7. Book Guided Package

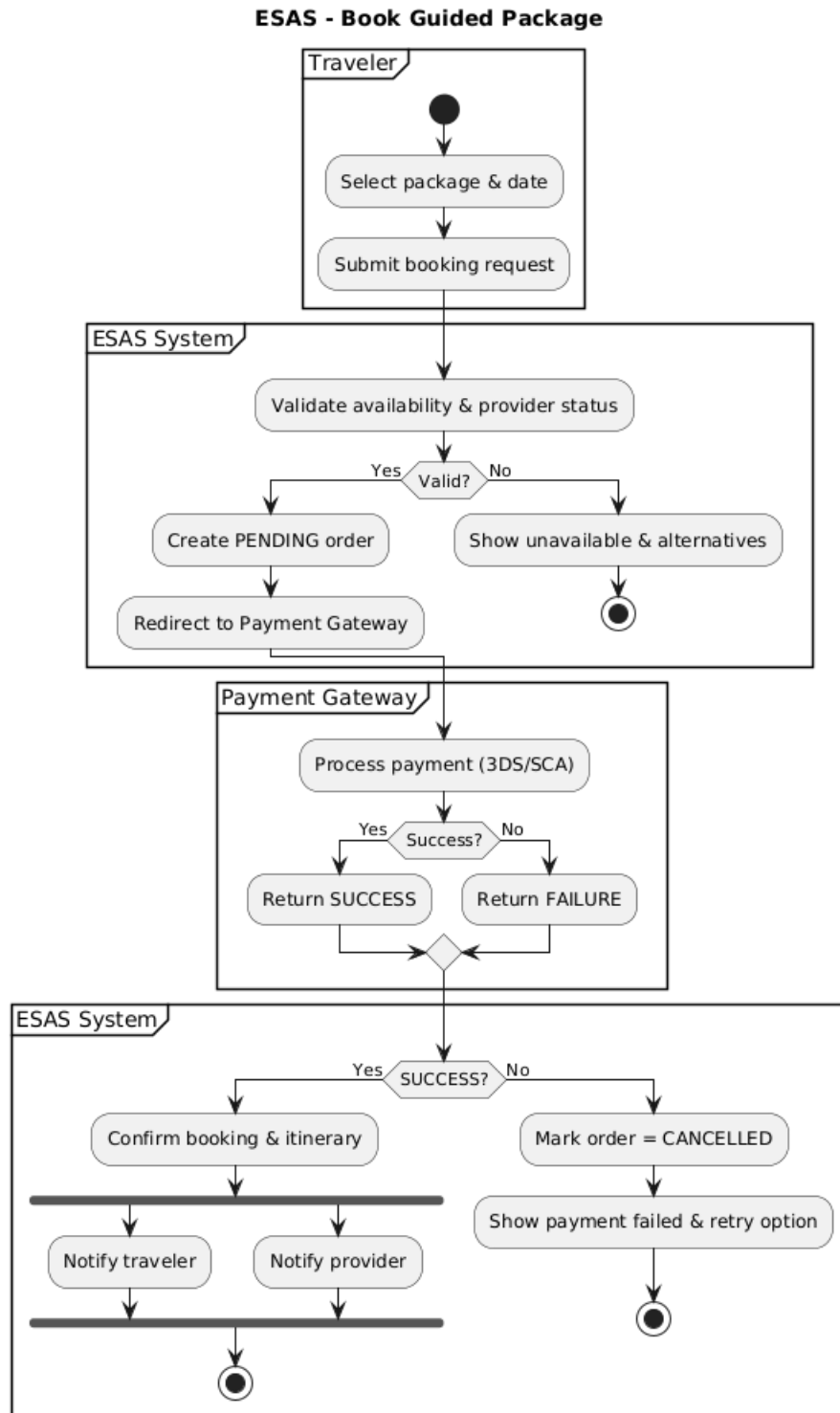


Figure 3-19: Book Guided Package Activity Diagram

3.2.10.8. Payment Flow

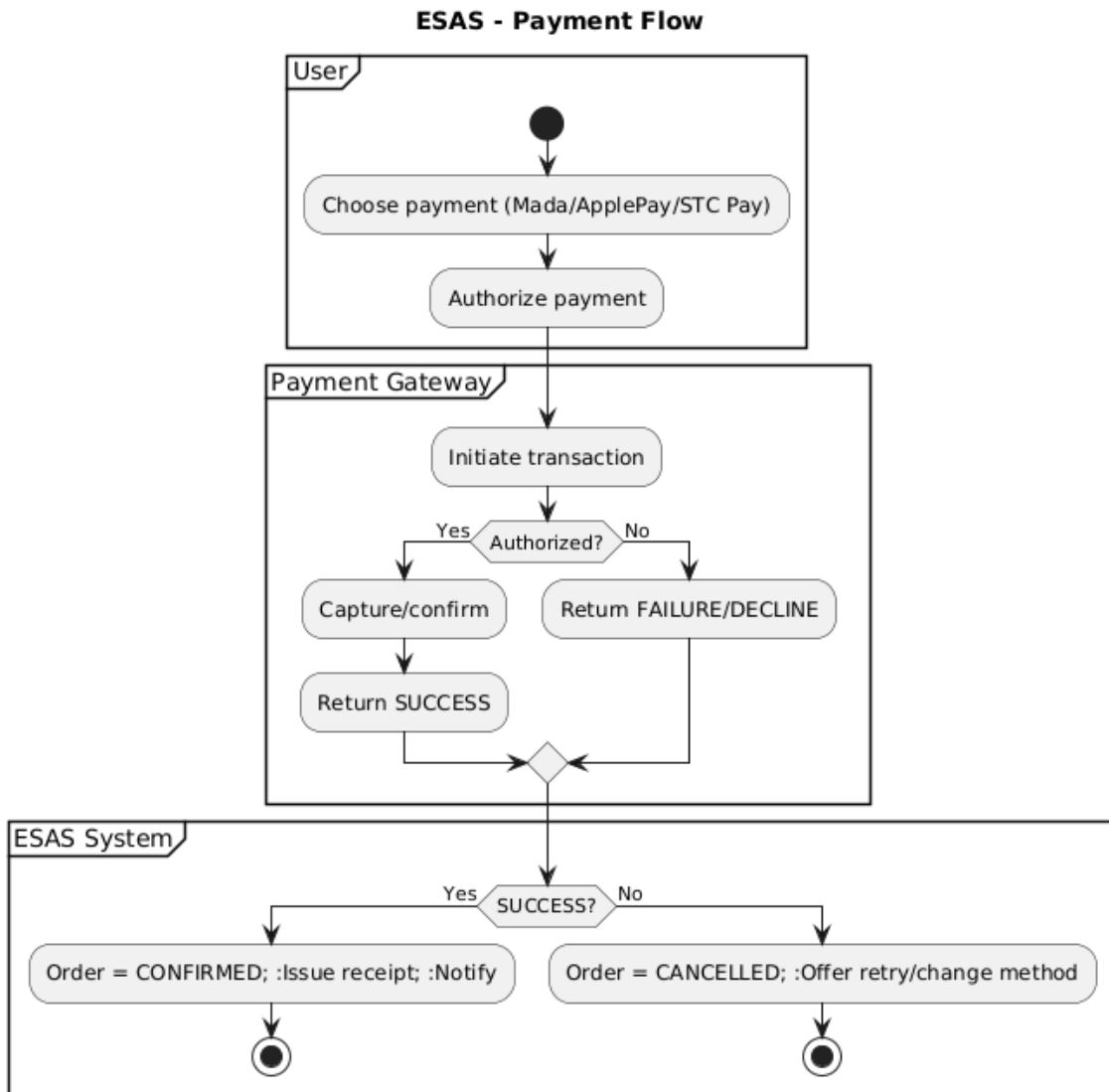


Figure 3-20: Payment Flow Activity Diagram

3.2.10.9. Limited Offer

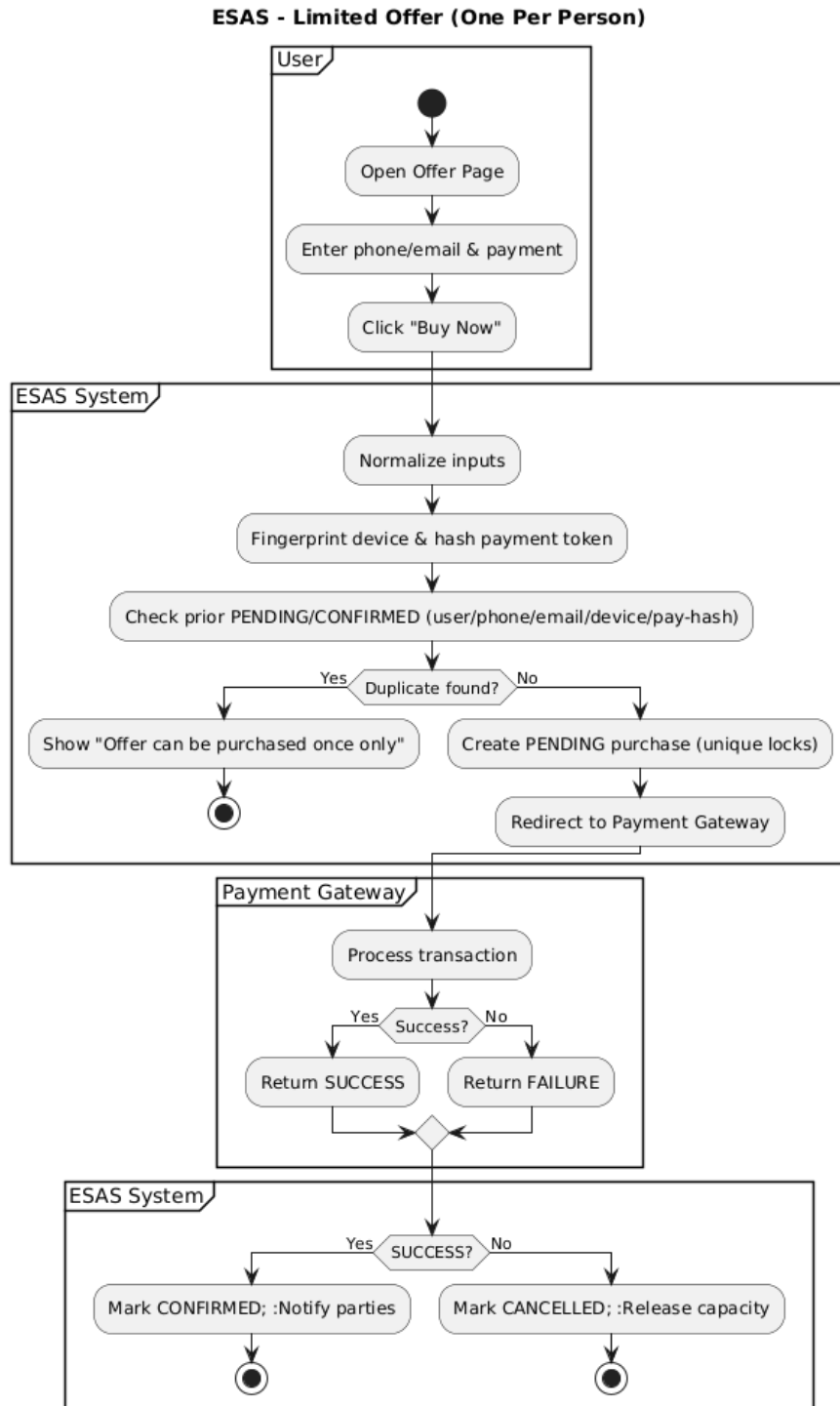


Figure 3-21: Limited Offer Activity Diagram

3.2.10.10. Provider Manage Offer

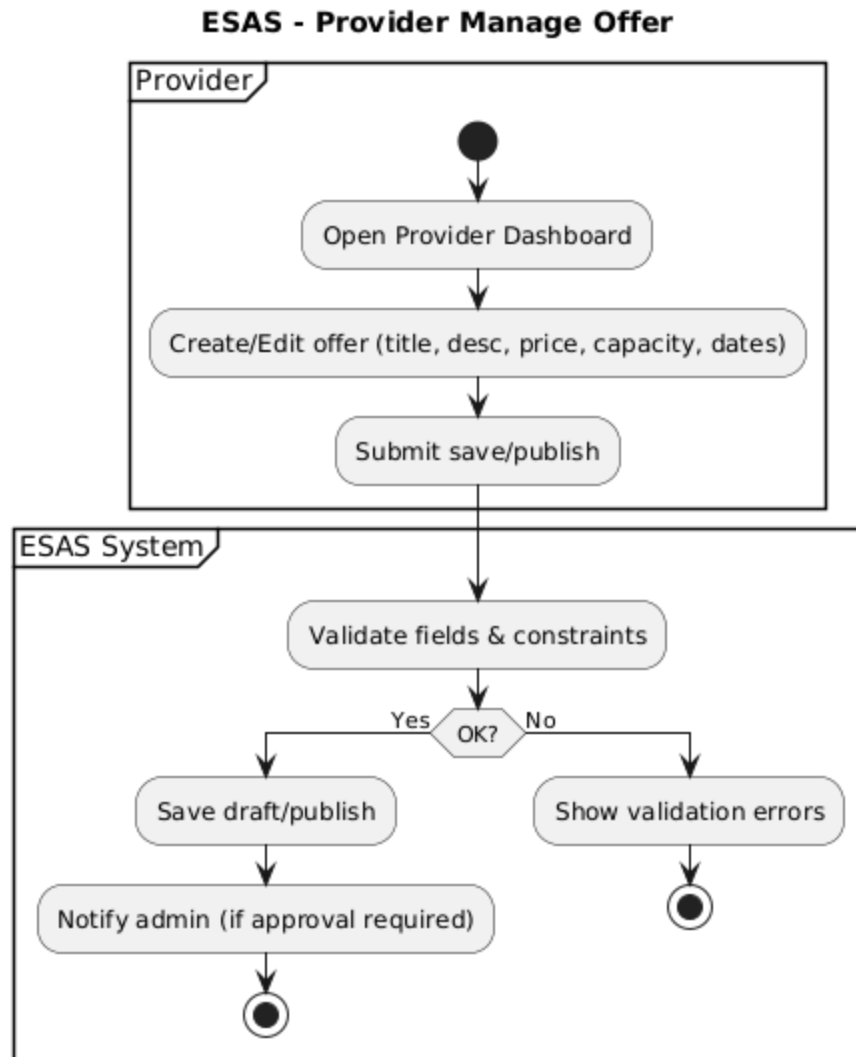


Figure 3-22: Provider Manage Offer Activity Diagram

3.2.10.11. Provider Availability & Pricing

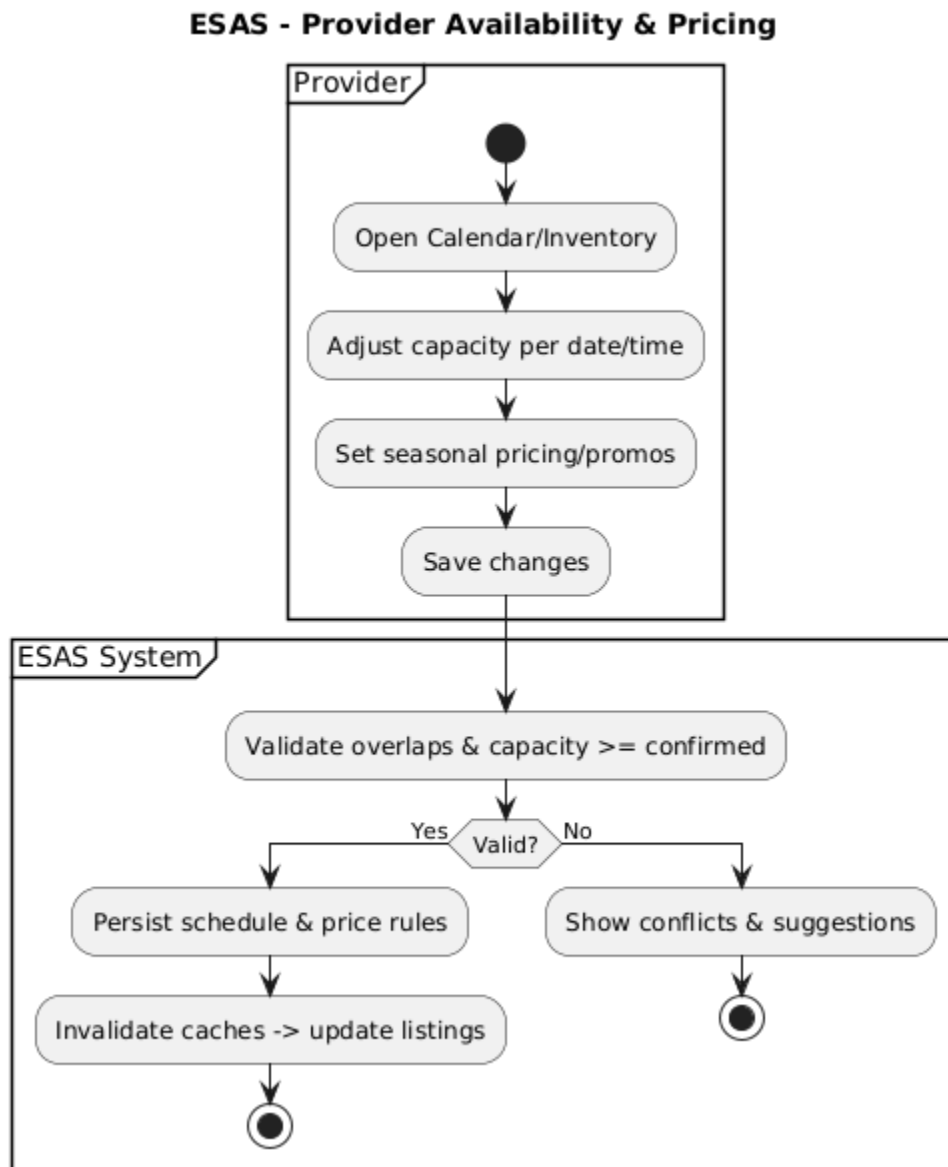


Figure 3-23: Provider Availability & Pricing Activity Diagram

3.2.10.12. Admin Approve Provider

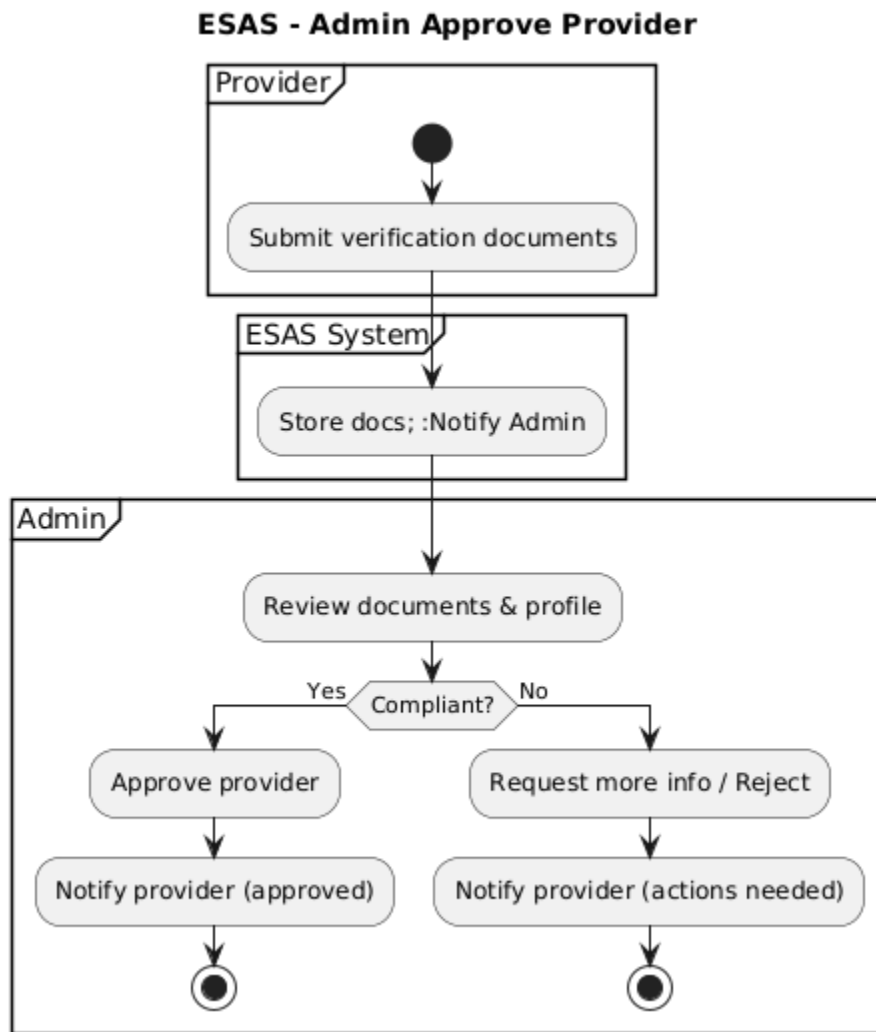


Figure 3-24: Admin Approve Provider Activity Diagram

3.2.10.13. Traveler Rate & Review

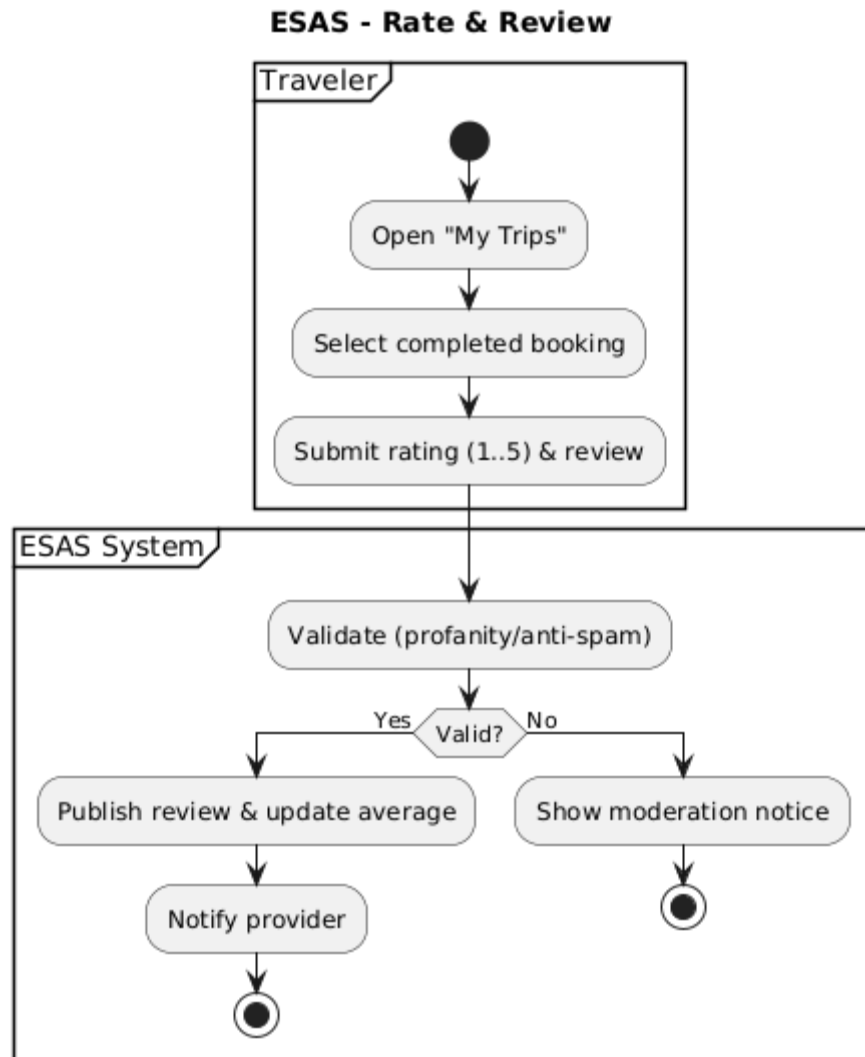


Figure 3-25: Traveler Rate & Review Activity Diagram

3.2.10.14. Cancel & Refund

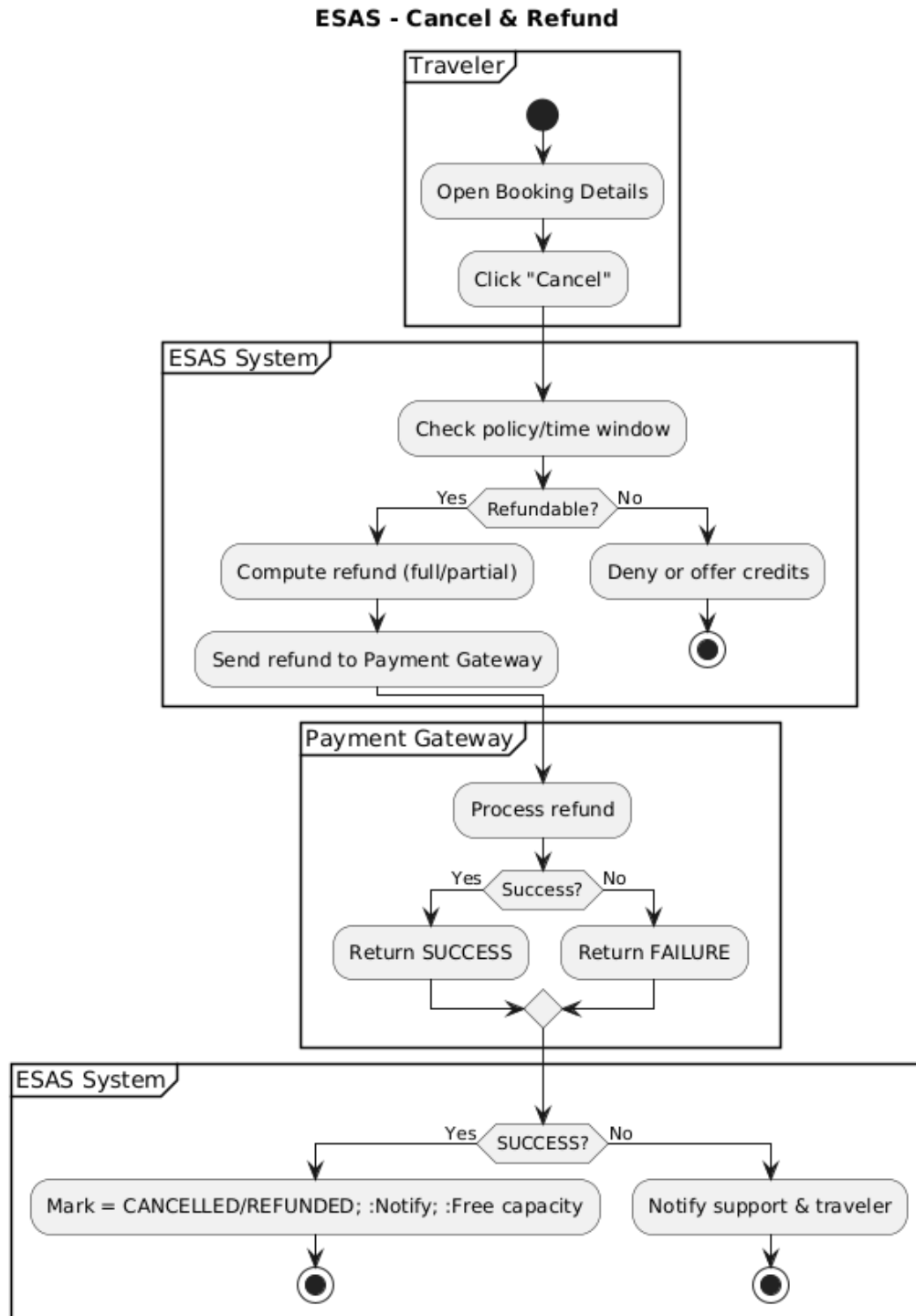


Figure 3-26: Cancel & Refund Activity Diagram

3.2.10.15. Notifications

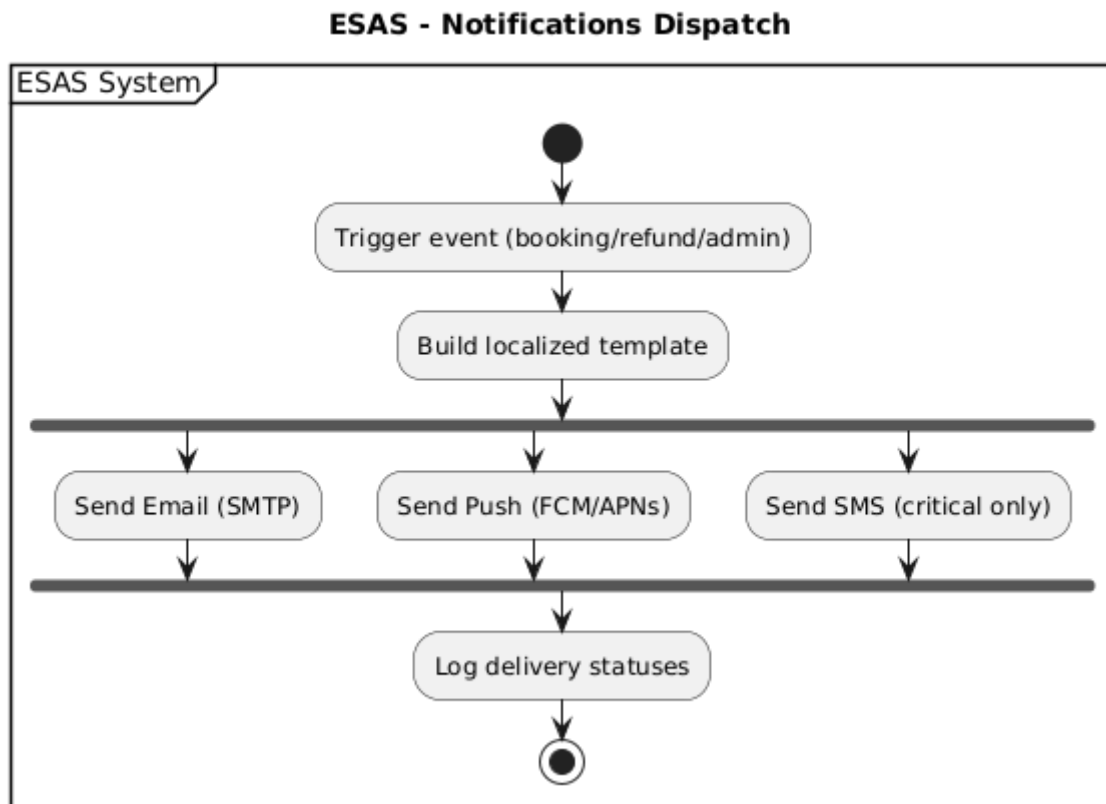


Figure 3-27: Notifications Activity Diagram

3.2.10.16. Provider Campaign Lifecycle

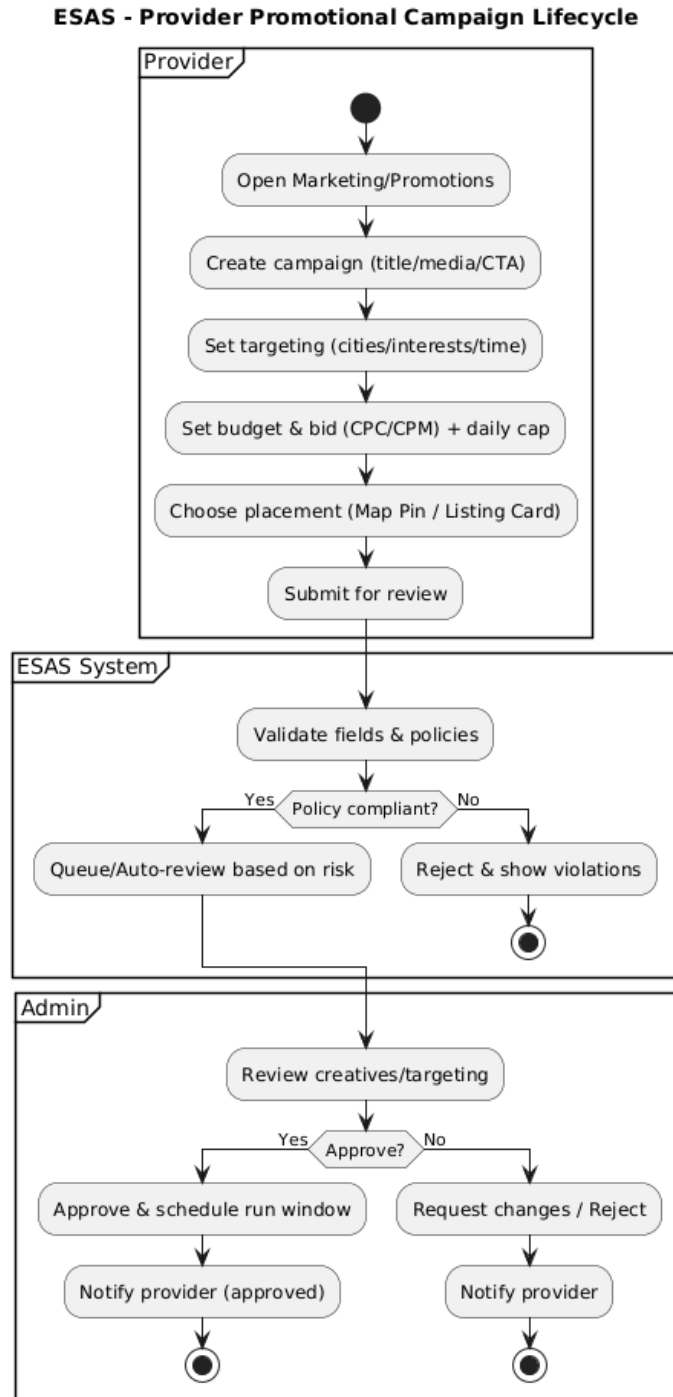


Figure 3-28: Provider Campaign Lifecycle Activity Diagram

3.2.10.17. Ad Serving, Billing, & Anti-Abuse

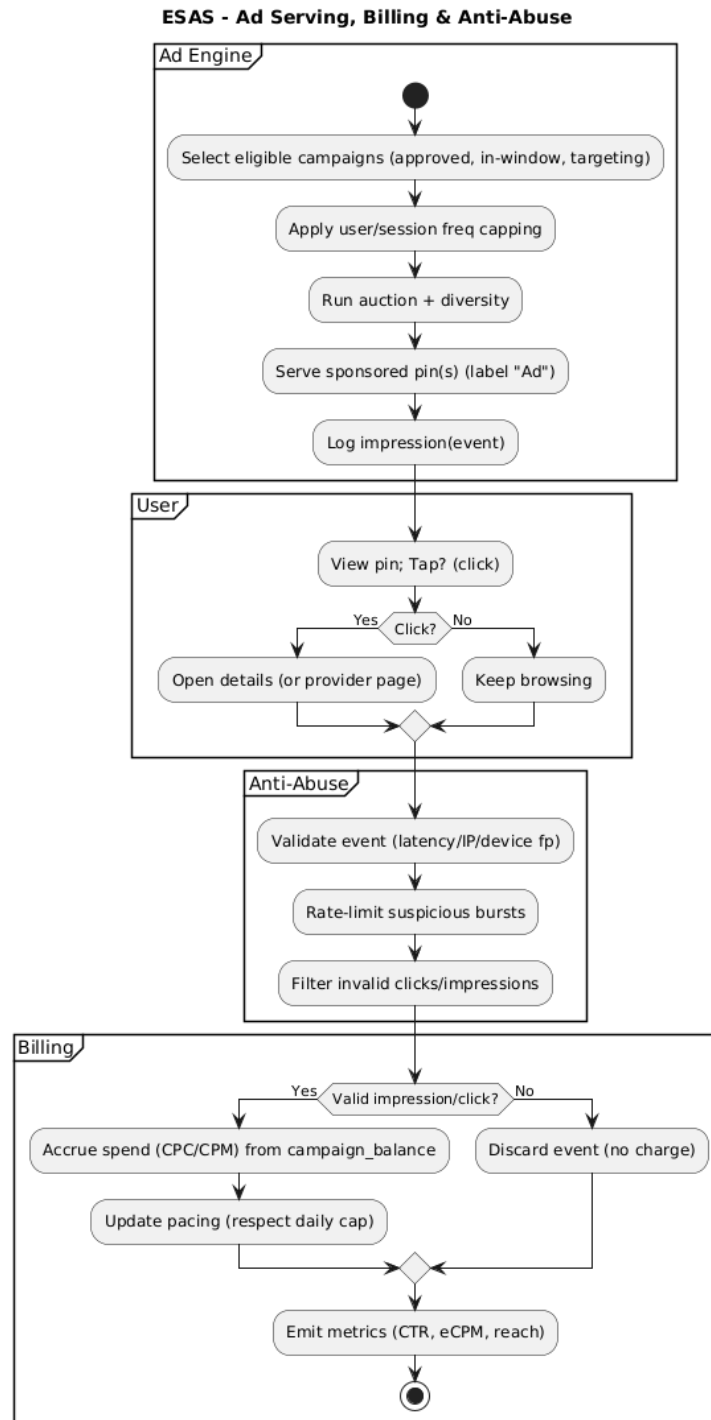


Figure 3-29: Ad Serving, Billing, & Anti-Abuse Activity Diagram

3.2.10.18. Password Reset

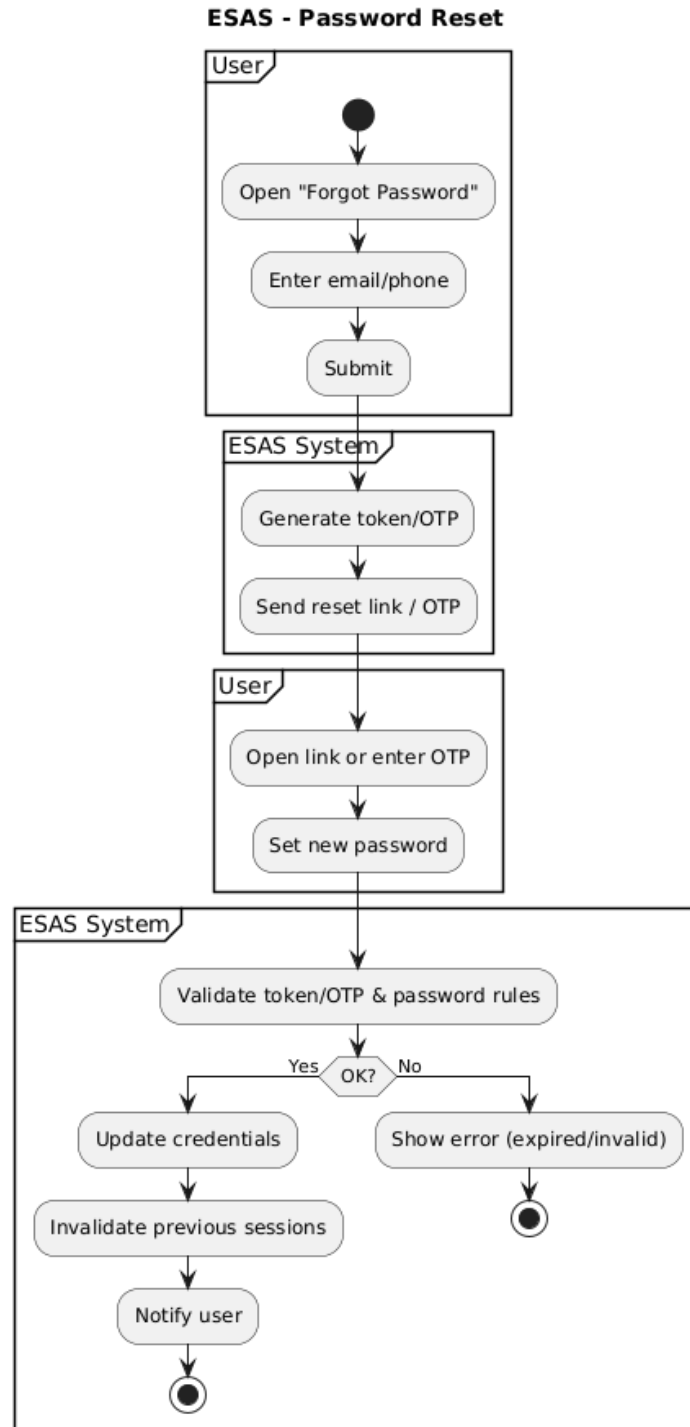


Figure 3-30: Password Reset Activity Diagram

3.2.11. Activity Diagram Description

3.2.11.1. Sign-Up

This diagram represents the process of registering a new account. The user provides details, accepts terms, and submits the form. The system validates data, checks for duplicates, and sends a one-time password (OTP). Once verified, the account is created, and the user is auto-signed in.

Design Note: This activity diagram highlights the system-controlled flow following user input, encompassing data validation, duplicate account checks, and OTP verification. The design ensures verified onboarding and prevents duplicate account creation while maintaining a secure and consistent registration process.

3.2.11.2. Sign-In

This diagram shows how users (Traveler, Provider, or Admin) log into the system. The user enters credentials, and the ESAS system validates them. If the login is successful, the system detects the user's role and redirects them to the corresponding dashboard. Otherwise, it shows an invalid credentials message.

Design Note: This activity diagram highlights role-based authentication and system-controlled validation of user credentials. Upon successful verification, the system determines the user's role and redirects the user to the appropriate dashboard, while invalid login attempts are safely rejected.

3.2.11.3. Progressive Onboarding

This diagram depicts the “browse-first” experience. Users can explore as guests without signing in. When they attempt to book, the system prompts them to sign in, sign up, or continue as a guest. Guest sessions are temporarily stored and merged later when they authenticate.

Note: Enhances accessibility by allowing seamless transition from guest to registered user.

3.2.11.4. Browse & Filter

This diagram outlines how users explore experiences using category, location, and season filters. The ESAS system retrieves filtered results, ranks them, and displays them on both

a map and a list interface. Users can refine filters or open experience details freely without booking.

Note: Focuses on exploration rather than conversion, supporting a discovery-first UX.

3.2.11.5. Map Explore

This diagram explains how ESAS merges organic (free) and sponsored (paid) pins on the interactive map. The system applies targeting, frequency capping, and provider diversity rules to ensure fair visibility. Users can view, save, share, or compare experiences and proceed to booking if desired.

Note: Balances user experience and monetization by blending sponsored content transparently.

3.2.11.6. Smart Roadmap

This diagram represents how ESAS generates intelligent travel roadmaps. The system fetches experiences, events, and sponsored stops matching the selected time and location. It scores them based on proximity, popularity, and user preferences, then creates a personalized daily route.

Note: Combines practical experiences with seasonal events and promotional content into one adaptive itinerary.

3.2.11.7. Book Guided Package

This diagram shows how travelers book guided experiences. The system checks availability, creates a pending booking, and redirects to payment. Upon success, both traveler and provider receive confirmation notifications.

Design Note: This activity diagram emphasizes transactional control over the booking process. The system validates availability and participant limits before initiating payment. A booking is confirmed only after successful payment authorization, while failure paths safely prevent partial or inconsistent reservations.

3.2.11.8. Payment Flow

This diagram details the process of payment authorization through gateways like Mada, Apple Pay, and STC Pay. It includes transaction initiation, confirmation, failure handling, and refund logic.

Design Note: This activity diagram is designed to ensure transactional safety during payment processing. The system interacts with the payment gateway to authorize transactions, handles success and failure responses, and prevents duplicate or incomplete payments through verification and confirmation mechanisms.

3.2.11.9. Limited Offer

This diagram shows how the system enforces single-purchase limits on limited offers. Before a payment, ESAS checks the user's identity and device fingerprints to prevent multiple purchases.

Note: Prevents external resale or abuse by restricting repeat transactions.

3.2.11.10. Provider Manage Offer

This diagram depicts how providers create or edit offers. After validation, the system saves the offer as a draft or publishes it. If the offer needs approval, the system notifies admins automatically.

Note: Ensures offer consistency and compliance with admin approval workflows.

3.2.11.11. Provider Availability & Pricing.

This diagram outlines how providers manage their schedule, capacity, and pricing. The system checks conflicts and validates that updates do not exceed confirmed bookings.

Note: Maintains up-to-date data accuracy and prevents overbooking.

3.2.11.12. Admin Approve Provider

This diagram shows the verification process for providers. Admins review submitted documents and either approve or reject providers. Notifications are automatically sent with required actions.

Design Note: This activity diagram reflects controlled administrative decision-making within the platform. Provider applications are evaluated against predefined verification criteria before approval or rejection. All decisions are logged to ensure accountability, traceability, and compliance with platform policies.

3.2.11.13. Rate & Review

This diagram illustrates how travelers submit ratings and reviews. The system validates content, prevents spam or profanity, updates averages, and notifies the provider.

Note: Improves platform credibility and encourages quality experiences.

3.2.11.14. Cancel & Refund

This diagram explains how travelers cancel bookings. The system checks refund eligibility according to policy, processes refunds through the payment gateway, and updates status records.

Design Note: This activity diagram illustrates policy-driven cancellation and refund handling. The system validates refund eligibility based on predefined rules before processing refunds through the payment gateway. Booking and payment states are updated consistently to ensure data integrity during cancellation scenarios.

3.2.11.15. Notifications Dispatch

This diagram describes how the system sends notifications (email, push, SMS) triggered by events such as booking, refund, or approval.

Note: Provides timely and automated communication across all channels.

3.2.11.16. Provider Campaign Lifecycle

This diagram shows how providers create promotional campaigns. They define targeting, budget, and duration. The system validates policy compliance, then forwards the campaign for admin review and approval.

Note: Ensures advertising integrity and compliance before publication.

3.2.11.17. Ad Serving, Billing & Anti-Abuse

This diagram represents how ESAS delivers ads fairly. It enforces frequency caps, filters invalid clicks, validates events, and processes billing for genuine impressions only.

Note: Protects both providers and users through transparent, fraud-resistant ad serving.

3.2.11.18. Password Reset

This diagram explains how users reset their passwords using OTP or token verification. Once verified, a new password is set, and all old sessions are invalidated.

Note: Enhances account recovery security by enforcing token-based verification.

3.3. External Interface Requirements

3.3.1. User Interface

The ESAS platform provides a bilingual (Arabic and English) web-based interface designed for both desktop and mobile users. The home page displays an interactive map that highlights Saudi destinations and experiences. Users can navigate using a top menu that includes sections such as Home, Experiences, Packages, Guides, My Trips, and Profile.

For **Travelers**, the interface is simple and visual, showing large experience cards, filters by category (cultural, adventure, coastal, family), and clear buttons such as “Book Now” or “Add to Trip.” For **Providers** (Saudi guides or experience owners), the interface allows creating and managing offerings, setting schedules, prices, and responding to traveler reviews. For **Administrators**, the interface includes dashboards for reviewing providers, moderating content, and generating reports.

The user interface follows accessibility standards: readable fonts, clear color contrast for outdoor use, and icon-based navigation. It also supports right-to-left layout for Arabic and left-to-right for English.

3.3.2. Hardware Interface

The system runs on standard devices and does not require any specialized hardware. On the client side, users can access ESAS from modern smartphones (Android 10+ and iOS

14+), tablets, or desktop browsers with internet access. GPS and location permissions are recommended to display nearby experiences on the map.

On the server side, the application will be hosted on Linux (Ubuntu 22.04+) or Windows Server using Spring Boot's embedded Tomcat server or a reverse proxy such as Nginx. Optional hardware includes mobile devices capable of receiving push notifications through system notification services.

3.3.3. Software Interfaces

ESAS interacts with several software components that support the platform's operation. The backend communicates with PostgreSQL for persistent storage and uses Flyway to manage database migrations. The frontend communicates with the backend through REST APIs using JSON. Spring Security and JWT handle authentication and authorization. The system may integrate with external services in future versions, such as map providers, weather APIs, payment gateways, email/SMS services, cloud storage, and notification services.

1. Frontend–Backend Interface

The Flutter frontend communicates with Spring Boot REST APIs using JSON request and response bodies. The frontend sends authentication tokens in the Authorization header for protected endpoints.

2. Authentication Interface

The frontend stores and sends JWT access tokens in Authorization headers. Refresh tokens are supported for session renewal. BCrypt is used for password hashing.

3. Database Interface

The backend communicates with PostgreSQL through Spring Data JPA and Hibernate. Flyway manages database schema versioning and seed data.

4. Static Media Interface

Static demonstration experience images are served by the backend from /demo/experiences/ under the static resources directory. The security configuration permits public access to these paths.

5. Future External Interfaces

The following software interfaces are documented as future/planned integrations and are not implemented in the current prototype:

- Payment gateways: HyperPay, Tap, or Moyasar (Mada, Apple Pay, STC Pay).
- Map and geolocation services: Google Maps JavaScript SDK, Directions API.
- Weather services: NCM (National Center for Meteorology) API or OpenWeather API.
- Email and notification services: SMTP, Firebase Cloud Messaging (FCM).
- Cloud object storage: Azure Blob, AWS S3, or MinIO for media files.

3.3.4. Communication Interface

The following external communication integrations are planned for future production versions and are not fully implemented in the current prototype:

1. External Integrations

1.1. Payment Gateways (HyperPay / Tap / Moyasar)

Future production versions may integrate with Saudi-approved payment gateways using:

- Hosted Payment Page (HPP) redirection
- 3D Secure (3DS2) mandatory for Mada cards
- HMAC or SHA-256 signature verification
- Asynchronous callback/webhook endpoints implemented in Spring Boot controllers

2. Maps & Location Services

- Google Maps JavaScript SDK
- Google Directions API (routing, trip-time estimation)
- Reverse geocoding API
- Geolocation based on browser permissions

3. Weather Services

- Weather data retrieved via NCM (National Center for Meteorology) API or OpenWeather API.
- Integrated via Spring's RestTemplate or WebClient.
- Weather conditions affect roadmap generation and experience filtering.

4. Email & Notification Services

- Emails sent via SMTP or cloud email gateways using Spring Mail.
- Push notifications sent using Firebase Cloud Messaging (FCM).

3.3.5. Use Case Diagram

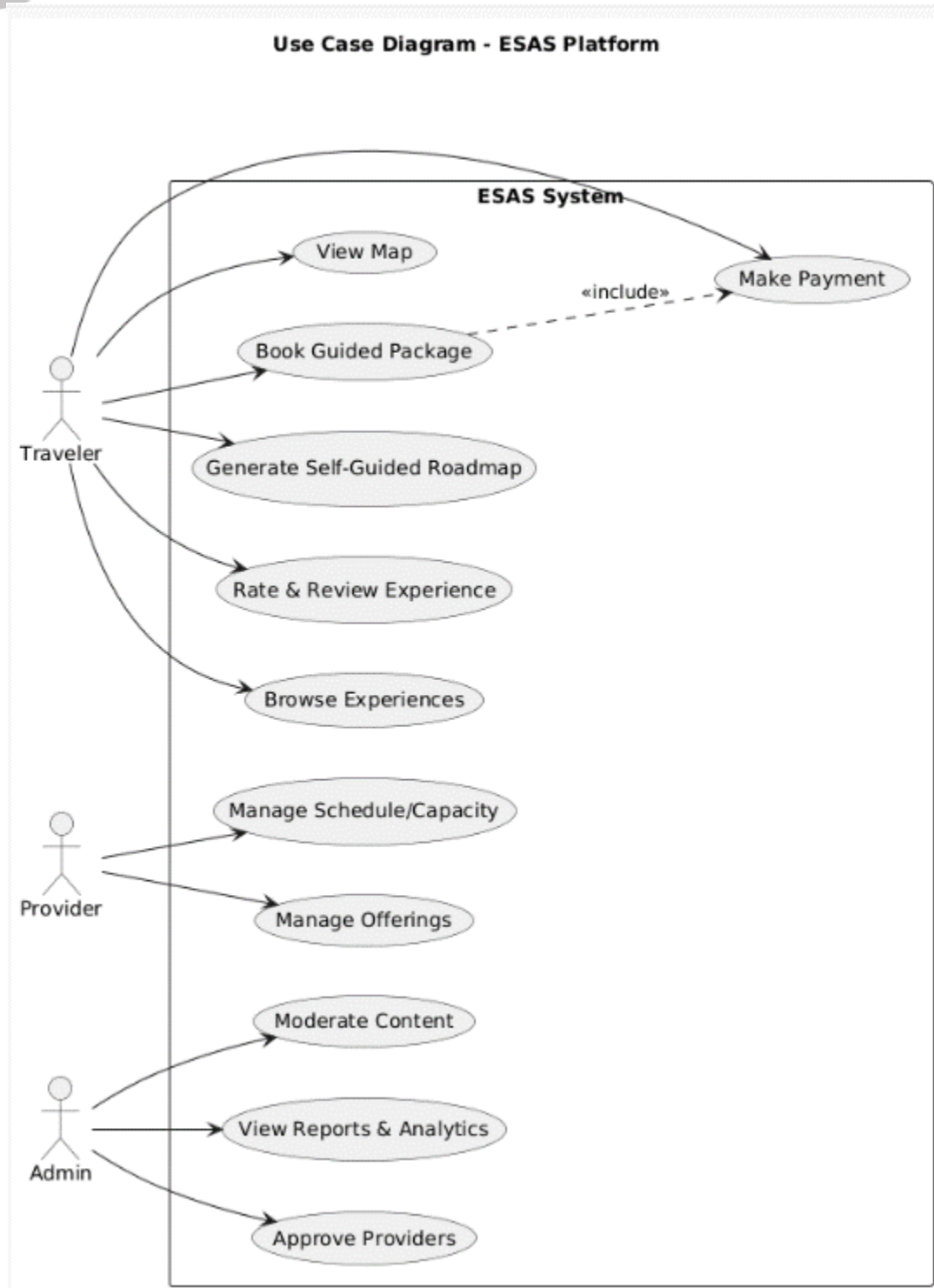


Figure 3-31: Use Case Diagram

3.3.6. Use Case Diagram Description

- **Traveler (Visitor):**

This actor represents tourists or visitors who use ESAS to explore Saudi experiences. Travelers can browse experiences, view the map, book guided packages, generate self-guided roadmaps, make secure payments, and rate or review completed trips.

- **Provider (Saudi Guide / Experience Owner):**

This actor represents verified local providers or guides who offer experiences on the platform. Providers can manage their offerings, update schedules, change prices, and view bookings made by travelers.

- **Administrator:**

This actor represents the system's operational and quality control team. Administrators review and approve new providers, moderate user content such as reviews or posts, and generate system-wide analytics or reports.

- **Included Use Cases:**

“Make Payment” is included in “Book Guided Package” because every booking must complete payment through the system. “View Map” supports “Browse Experiences” by allowing users to visually explore available locations.

3.4. System Features

This section defines the functional requirements of the ESAS system, organized by feature area. Each feature includes a description, stimulus/response sequences, and formal functional requirements. Features are labeled by their current prototype status: Implemented, Partially Implemented, or Future Work.

3.4.1. User Authentication and Role Management

Description and Priority

This high-priority feature allows users to access the system according to their assigned role. The system supports Traveler, Provider, and Administrator roles. Authentication protects sensitive actions such as cart operations, booking, provider management, and administrative approval. Status: Implemented.

Stimulus/Response Sequences

1. User submits login credentials.
2. System validates credentials.
3. System generates an authentication token if credentials are valid.
4. System grants access according to the user role.
5. If credentials are invalid, the system displays a safe error message.

Functional Requirements

- REQ-AUTH-001: The system shall allow registered users to authenticate using valid credentials.
- REQ-AUTH-002: The system shall allow user signup according to supported account types.
- REQ-AUTH-003: The system shall issue JWT access tokens for authenticated users.
- REQ-AUTH-004: The system shall support refresh/logout behavior according to the implemented authentication flow.
- REQ-AUTH-005: The system shall assign users one or more roles such as Traveler, Provider, or Admin.
- REQ-AUTH-006: The system shall protect restricted APIs using JWT-based authentication.
- REQ-AUTH-007: The system shall prevent users from accessing functions outside their assigned role.
- REQ-AUTH-008: The system shall store passwords using secure hashing.
- REQ-AUTH-009: The system shall return safe error messages for failed authentication attempts.

3.4.2. Public Catalog and Experience Browsing

Description and Priority

This high-priority feature allows guests and travelers to browse approved Saudi experiences and guided packages through backend-driven catalog data. Status: Implemented.

Stimulus/Response Sequences

1. User opens the catalog or home page.
2. Frontend requests available experiences from the backend.
3. Backend returns approved public content.
4. Frontend displays the results.
5. If no results are available, the frontend displays an empty state.

Functional Requirements

- REQ-CAT-001: The system shall display public experiences retrieved from the backend.
- REQ-CAT-002: The system shall only display approved and non-deleted experiences in the public catalog.
- REQ-CAT-003: The system shall allow users to view experience summary information such as title, city, category, price, and image.
- REQ-CAT-004: The system shall support public guided package browsing according to the implemented backend scope.
- REQ-CAT-005: The system shall display an empty state when no experiences are available.
- REQ-CAT-006: The system shall display a safe error message if catalog data cannot be loaded.

3.4.3. Experience Details and Filtering

Description and Priority

This high-priority feature allows users to view details of approved experiences and use supported filtering/search options. Status: Partially Implemented.

Stimulus/Response Sequences

1. User selects an experience or enters filter criteria.
2. Frontend sends a request to the backend.
3. Backend returns matching approved results or selected details.
4. Frontend displays the result.
5. If the item is unavailable or unapproved, the system prevents booking actions.

Functional Requirements

- REQ-EXP-001: The system shall allow users to view the details of a selected approved experience.
- REQ-EXP-002: The system shall allow users to filter experiences by supported criteria such as keyword, city, region, category, price, or sort option.
- REQ-EXP-003: The system shall retrieve filtering results from the backend rather than relying on frontend-only filtering for real domain data.
- REQ-EXP-004: The system shall prevent unavailable, deleted, or unapproved experiences from being booked.
- REQ-EXP-005: The system shall display localized labels and messages where applicable.

3.4.4. Traveler Cart

Description and Priority

This high-priority feature allows authenticated travelers to manage selected experiences or packages before checkout. Status: Implemented.

Stimulus/Response Sequences

1. An authenticated traveler selects an eligible item.
2. System validates item eligibility.
3. The item is added to the backend cart.
4. Traveler views or updates cart contents.
5. System reflects cart changes through backend APIs.

Functional Requirements

- REQ-CART-001: The system shall allow authenticated travelers to add eligible experiences or packages to their cart.
- REQ-CART-002: The system shall store cart data in the backend.
- REQ-CART-003: The system shall allow travelers to view their cart.
- REQ-CART-004: The system shall allow travelers to remove items from the cart.
- REQ-CART-005: The system shall prevent users from adding unavailable, unapproved, or deleted items to the cart.

- REQ-CART-006: The system shall display safe localized error messages for failed cart operations.

3.4.5. Simulated Checkout and Booking

Description and Priority

This high-priority feature allows authenticated travelers to complete a simulated checkout flow for demonstration purposes. Real payment gateway integration is future work. Status: Implemented.

Stimulus/Response Sequences

1. Traveler reviews cart.
2. Traveler starts checkout.
3. System simulates payment confirmation.
4. System creates booking records.
5. System clears the cart after successful simulated checkout.

Functional Requirements

- REQ-CHECK-001: The system shall allow authenticated travelers to perform simulated checkout for eligible cart items.
- REQ-CHECK-002: The system shall create booking records after successful simulated checkout.
- REQ-CHECK-003: The system shall clear the traveler cart after successful simulated checkout.
- REQ-CHECK-004: The system shall maintain booking history for the traveler.
- REQ-CHECK-005: The current prototype shall not claim real payment processing.
- REQ-CHECK-006: The system shall support future integration with certified local payment gateways.

3.4.6. Booking Management

Description and Priority

This high-priority feature allows travelers and relevant system roles to view and manage bookings according to the current prototype scope. Status: Implemented/Partially Implemented.

Functional Requirements

- REQ-BOOK-001: The system shall create bookings for approved, eligible experiences or packages.
- REQ-BOOK-002: The system shall allow travelers to view their booking history.
- REQ-BOOK-003: The system shall support booking cancellation according to the implemented prototype rules.
- REQ-BOOK-004: The system shall display booking payment status according to the implemented payment simulation.
- REQ-BOOK-005: The system shall provide demo receipt information where implemented.
- REQ-BOOK-006: The system shall protect booking data so users can access only authorized records.

3.4.7. Wishlist and Wallet

Description and Priority

This medium-priority feature improves traveler convenience by supporting wishlist functionality and wallet/payment simulation views. Status: Implemented/Partially Implemented.

Functional Requirements

- REQ-WISH-001: The system shall allow authenticated travelers to add eligible experiences to their wishlist.
- REQ-WISH-002: The system shall allow authenticated travelers to remove experiences from their wishlist.
- REQ-WISH-003: The system shall allow authenticated travelers to view their wishlist.
- REQ-WALLET-001: The system shall display wallet balance or transaction data according to the implemented prototype scope.

- REQ-WALLET-002: The system shall support simulated or wallet-based payment flows according to implemented prototype scope.
- REQ-WALLET-003: The system shall not describe wallet functionality as real bank settlement or production payment processing.

3.4.8. Provider Registration and Offering Management

Description and Priority

This high-priority feature allows providers to onboard, submit verification information, and manage their own offerings according to the current prototype scope. Status: Partially Implemented.

Functional Requirements

- REQ-PROV-001: The system shall allow providers to request registration or onboarding.
- REQ-PROV-002: The system shall support provider verification submission.
- REQ-PROV-003: The system shall require admin approval before provider offerings become publicly visible.
- REQ-PROV-004: The system shall allow approved/verified providers to create, update, and submit their own experiences.
- REQ-PROV-005: The system shall prevent providers from editing offerings owned by other providers.
- REQ-PROV-006: The system shall mark submitted offerings as requiring admin review before public publication.
- REQ-PROV-007: Advanced provider payouts, campaigns, and advertisement tools are future work.

3.4.9. Admin Approval, Moderation, and Audit

Description and Priority

This high-priority feature allows administrators to control provider approval, experience publication, and audit functions. Status: Implemented.

Functional Requirements

- REQ-ADM-001: The system shall allow administrators to review provider verification requests.
- REQ-ADM-002: The system shall allow administrators to approve or reject provider requests.
- REQ-ADM-003: The system shall allow administrators to review submitted experiences before publication.
- REQ-ADM-004: The system shall allow administrators to approve or reject submitted experiences.
- REQ-ADM-005: The system shall maintain administrative actions for audit purposes.
- REQ-ADM-006: The system shall restrict administrative functions to authorized admin users only.
- REQ-ADM-007: Advanced dispute/refund/analytics dashboards are future work.

3.4.10. Reviews and Ratings

Description and Priority

This medium-priority feature is part of the proposed full system. The current prototype does not confirm a complete runtime review API/UI implementation. Status: Documented/Schema Only (Future Work).

Functional Requirements

- REQ-REV-001: The system shall allow travelers to submit ratings and reviews after completing a booking when this feature is implemented.
- REQ-REV-002: The system shall associate reviews with the correct traveler and experience.
- REQ-REV-005: The current prototype shall not claim complete runtime reviews of the API/UI.

3.4.11. Localization and User Interface

Description and Priority

This high-priority feature supports usability for Arabic and English users. Status: Implemented.

Functional Requirements

- REQ-UI-001: The system shall support English and Arabic user interface text.
- REQ-UI-002: The system shall support right-to-left layout behavior for Arabic.
- REQ-UI-003: The system shall support left-to-right layout behavior for English.
- REQ-UI-004: The system shall provide clear loading, empty, error, and retry states.
- REQ-UI-005: The system shall provide responsive layouts suitable for web and Android demonstration.
- REQ-UI-006: The frontend shall consume backend APIs for real domain data rather than hardcoding authoritative domain data.

3.4.12. Notifications and Communication

Description and Priority

This medium-priority feature is part of the proposed full system. Complete runtime notification delivery/API/UI is not confirmed in the current prototype. Status: Documented/Schema Only (Future Work).

Functional Requirements

- REQ-NOTIF-001: The system shall support notification concepts for booking, provider approval, moderation, and user communication.
- REQ-NOTIF-002: The current prototype shall not claim production email/SMS/push notification support.
- REQ-NOTIF-003: The system shall support future notification delivery through email, SMS, or push services.
- REQ-COMM-001: The system shall support a contact form for user inquiries. Backend contact support is planned/future work.

3.4.13. Smart Roadmaps, Recommendations, and Maps

Description and Priority

These features are part of the proposed full system vision. None are confirmed as runtime-implemented in the current prototype. Status: Documented/Schema/OpenAPI Only (Future Work).

Functional Requirements

- REQ-MAP-001: The system shall support interactive map exploration in future implementation.
- REQ-REC-001: The system shall support AI and weather-based recommendations in future implementation.
- REQ-ROAD-001: The system shall support self-guided roadmap generation in future implementation.
- REQ-ROAD-002: The system shall support offline roadmap downloads in future implementation.

3.4.14. Reports, Monitoring, and Audit

Description and Priority

This feature supports platform governance. Partial implementation exists for audit logs and admin reporting. Status: Partially Implemented.

Functional Requirements

- REQ-AUD-001: The system shall maintain audit logs for administrative actions.
- REQ-AUD-002: The system shall provide admin access to operational statistics and reports.
- REQ-AUD-003: Full analytics/dispute/refund dashboards are future work.

3.5. Non-Functional Requirements

3.5.1. Usability

The ESAS platform must be intuitive and visually clear to users with varying technical backgrounds. The design prioritizes simplicity, bilingual support (Arabic and English), and accessibility. Buttons, icons, and navigation paths should be self-explanatory. The platform will follow usability standards, ensuring consistent layouts, responsive design, and compatibility with multiple devices and screen sizes.

3.5.2. Reliability

System reliability is crucial for continuous availability and minimal downtime. The platform should maintain **99.5% uptime**, with automatic recovery mechanisms for server

failures. Data integrity must be maintained through daily backups, and transactions must be completed successfully even under peak traffic.

3.5.3. Security

Security is a top priority for protecting users' personal information and payments. The system uses HTTPS for all communications, hashed passwords, and JWT-based authentication. No sensitive financial data (like credit card numbers) will be stored on ESAS servers. The platform complies with Saudi cybersecurity and privacy standards.

3.6. Performance Requirements

The system must load main pages within **3 seconds** under standard network conditions. API response time should not exceed **800 ms** for 95% of requests. The platform should handle at least **500 concurrent users** without performance degradation. Database queries will be optimized and cached where applicable.

3.7. Safety Requirements

While ESAS does not address physical safety, it provides safety-related travel information, such as weather alerts, regional advisories, and emergency contact numbers. System safety includes secure session management and prevention of data loss through regular backups and version control.

3.8. Security Requirements

Access to administrative dashboards requires authentication and role-based authorization. User sessions will automatically expire after inactivity. Sensitive operations (refunds, deletions, payouts) require confirmation prompts. All API endpoints will validate inputs to prevent attacks such as SQL injection, XSS, and CSRF. The platform shall comply with the Saudi Personal Data Protection Law (PDPL). The platform should follow NCA Essential Cybersecurity Controls (ECC-1, ECC-2).

3.9. Software Quality Attributes

The ESAS system should demonstrate:

- **Scalability:** Easy to extend to new cities or experience categories.
- **Maintainability:** Clear modular structure for front-end and back-end.

- **Portability:** Runs on both local and cloud environments with minimal setup.
- **Reusability:** Shared service modules (auth, booking, payment) across web and mobile.
- **Localization:** Full support for Arabic and English with right-to-left adaptation.

3.10. Business Rules

- Only verified Saudi guides and providers may publish offers.
- Payments are processed only through approved local gateways.
- Bookings cannot be confirmed without successful payment.
- Users may cancel bookings according to predefined provider policies.
- Ratings and reviews can only be submitted after a completed booking.
- Admins may remove content that violates platform rules or laws.

3.11. Other Requirements

- System updates are deployed outside business hours to minimize disruption.
- Logs are retained for at least 90 days for audit purposes.
- Notifications are bilingual and formatted consistently.
- The platform must comply with **Vision 2030 digital transformation standards.**

Chapter 4: Software Design

4.1. Design Overview

4.1.1. Introduction

This chapter defines the structural and visual design of the ESAS platform and illustrates how system components interact to meet the specified requirements. The design focuses on clarity, maintainability, and alignment with usability and performance goals.

4.1.2. Environment Overview

The system design targets a modular, layered architecture using Flutter (Dart) as the frontend connected to a Spring Boot backend (Java 21). The environment supports REST APIs, JWT authentication, and PostgreSQL for persistence. Flyway manages database schema versioning. Development and local demonstration environments are configured using Docker Compose.

4.1.3. Constraints and Assumptions

- The system assumes a stable internet connection and modern browser/Android compatibility.
- Real third-party APIs (maps, payments, weather) are planned for future integration.
- Local Docker/PostgreSQL deployment is used for the prototype; production cloud deployment is future work.
- The design avoids proprietary tools that restrict portability.
- The interface and backend both comply with security and data handling policies.

4.1.4. System Architecture

ESAS uses a client-server architecture. The Flutter frontend acts as the client application, while the Spring Boot backend provides REST APIs and manages business logic, authentication, authorization, and persistence. PostgreSQL stores system data, and Flyway manages database migrations. The architecture separates presentation concerns from domain rules, ensuring that critical data and workflows remain controlled by the backend.

4.1.5. Interfaces and Data Stores

The ESAS system exposes REST API interfaces for frontend clients. These APIs support authentication, catalog browsing, experience details, guided package browsing, cart operations, simulated checkout, bookings, wishlist, wallet/payment simulation, provider workflows, and administrative workflows according to the implemented scope.

The main data store is the PostgreSQL database. It stores user accounts, roles, provider/admin data, destinations, experiences, guided packages, cart items, bookings, payment transaction records, wallet records, wishlist records, audit records, and other domain records according to the database schema.

4.2. User Interface Prototype

The following figures show the ESAS frontend prototype screens. These were developed using Figma and display the initial mock-up of the user interface.

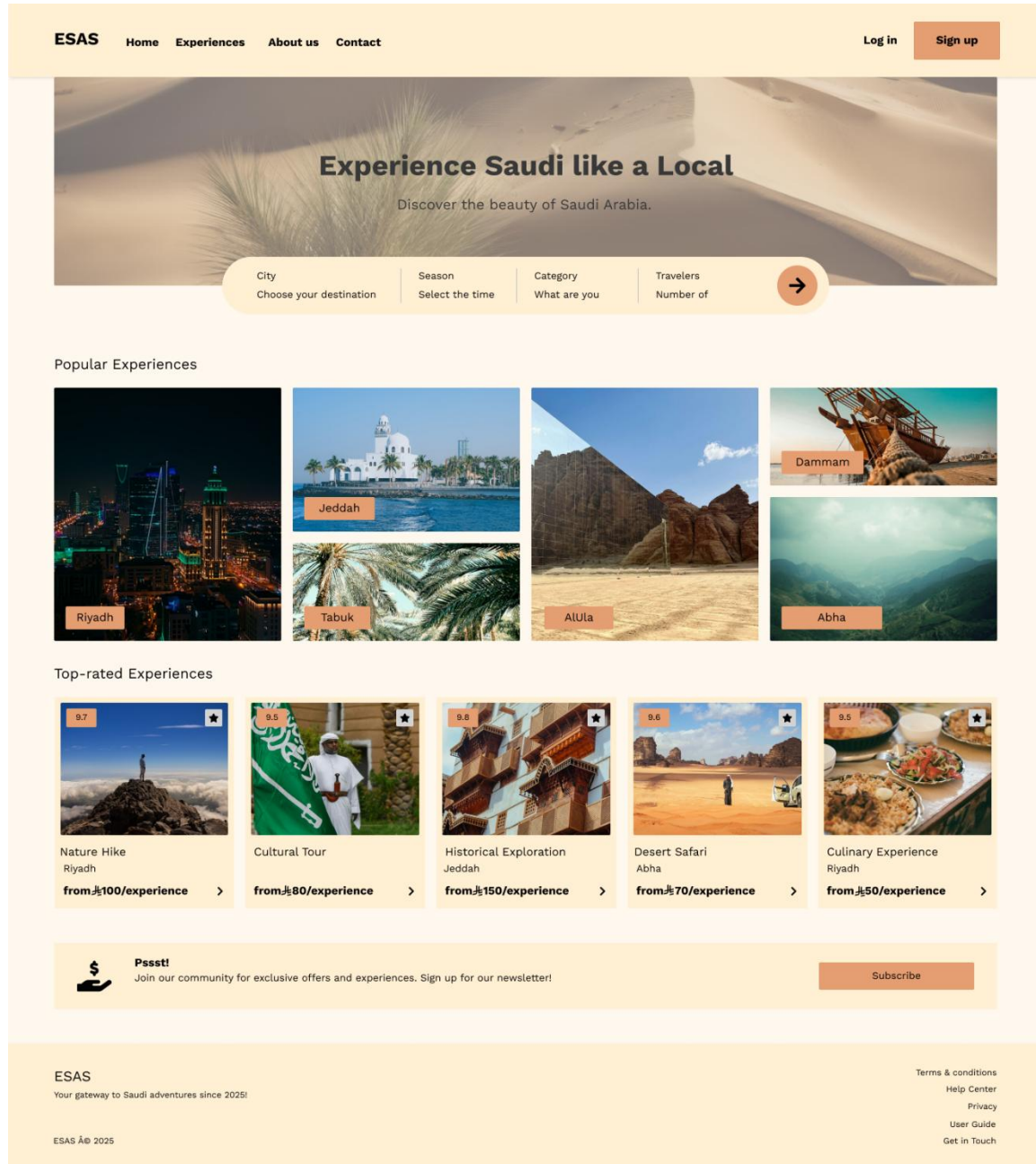
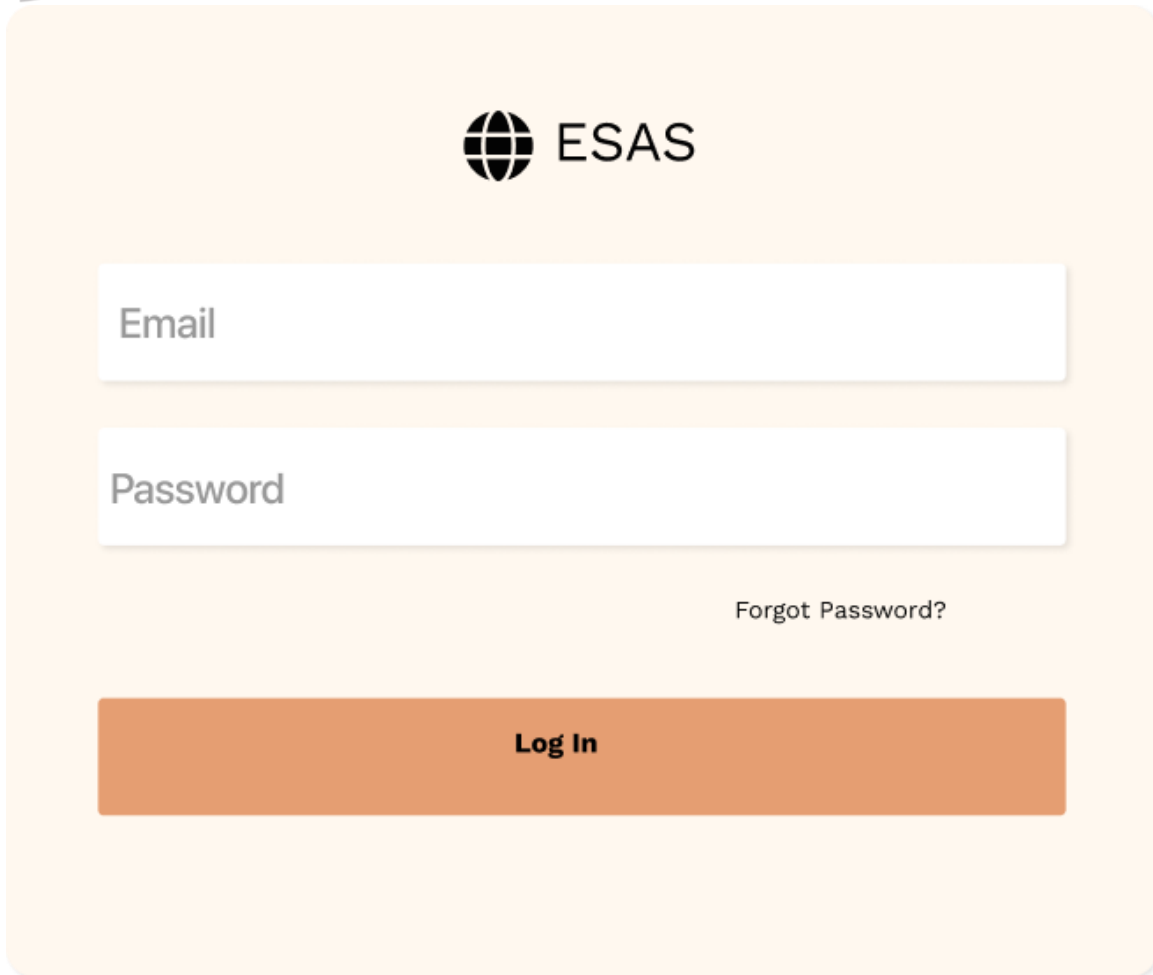


Figure 4-1: ESAS Demo Home Page



The login popup features the ESAS logo at the top center, which consists of a globe icon and the text "ESAS". Below the logo are two white input fields with rounded corners. The first field is labeled "Email" and the second is labeled "Password". To the right of the password field is a link that says "Forgot Password?". At the bottom of the popup is a large orange button with the text "Log In" in white.

Figure 4-2: ESAS Demo Login Popup


[Home](#)
[Experiences](#)
[About us](#)
[Contact](#)

Category 

Seasonal Highlights

Immerse in Saudi culture!



Season 

Guests 

Location 

Experience type 

Price 

Guided Tours	Rating	Date	Experience	Price
Desert Safari Adventure	5	3 nights		from SAR 700
Cultural Heritage Tour	5	5 nights		from SAR 550
City Explorer Package	4	7 nights		from SAR 1000
Beach Getaway	4	4 nights		from SAR 930
Historical Sites Tour	5	6 nights		from SAR 1200
Local Cuisine Experience	4	2 nights		from SAR 550
Adventure Awaits	3	5 nights		from SAR 480
Luxury Retreat	5	4 nights		from SAR 930

ESAS
 Your gateway to Saudi adventures since 2025!

Terms & conditions
 Help Center
 Privacy
 User Guide
 Get in Touch

ESAS Â© 2025

Figure 4-3: ESAS Demo Guided Packages Preview Page

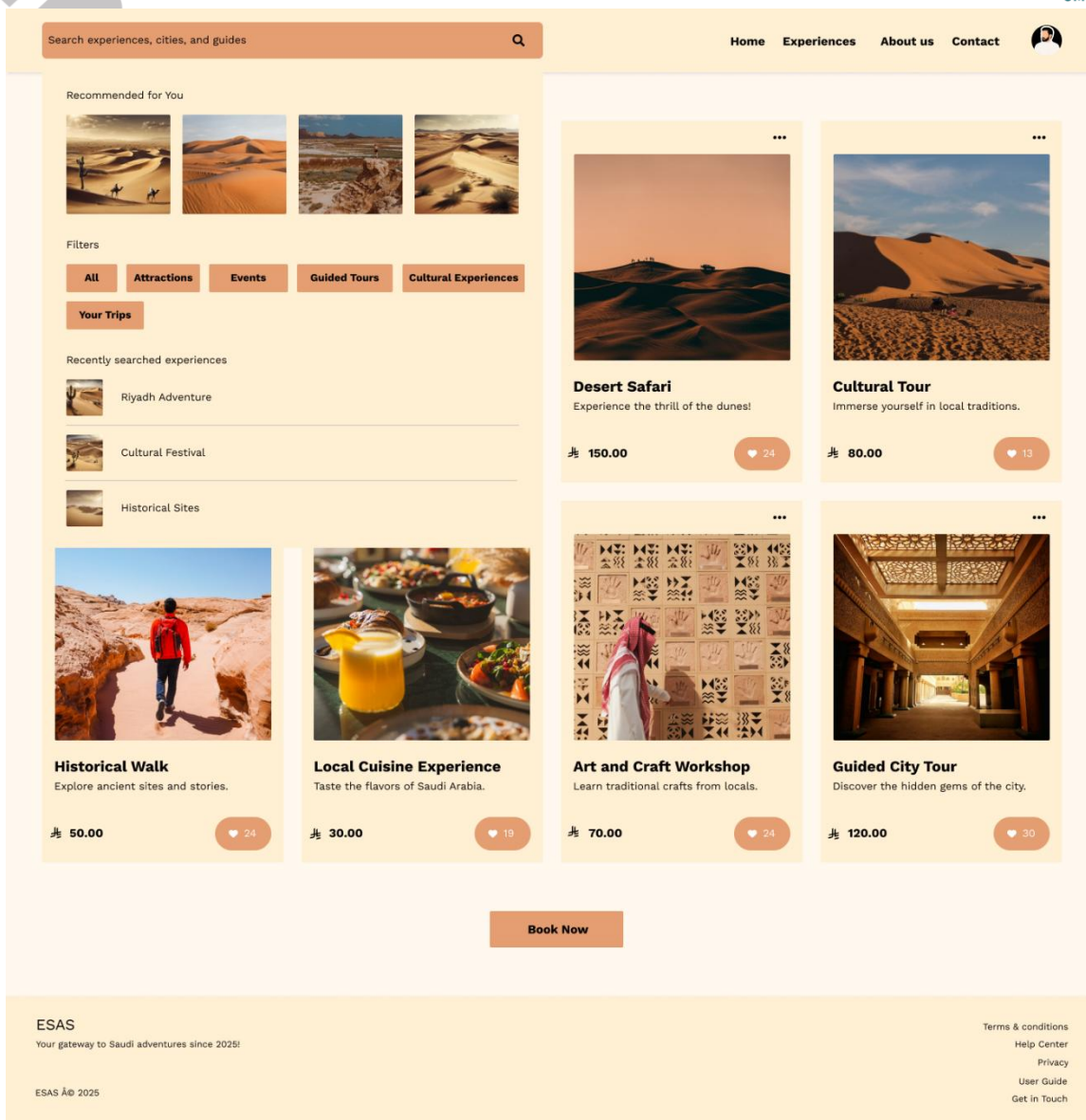


Figure 4-4: ESAS Demo Experience Search Page

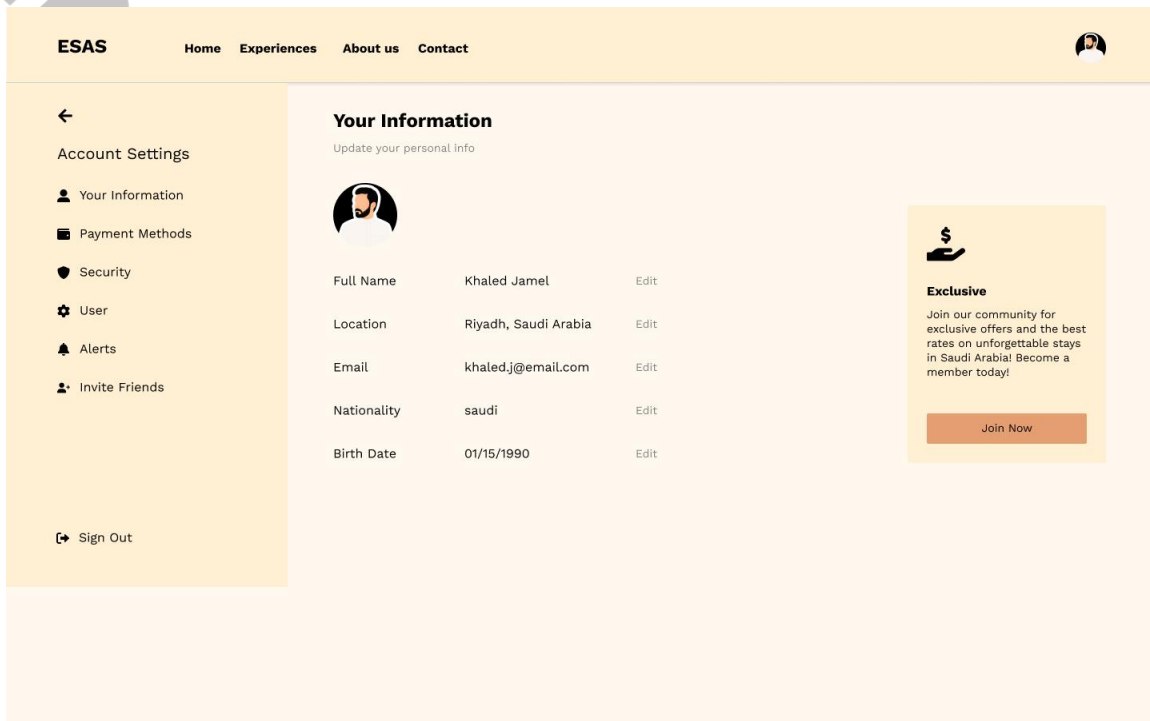


Figure 4-5: ESAS Demo User Profile

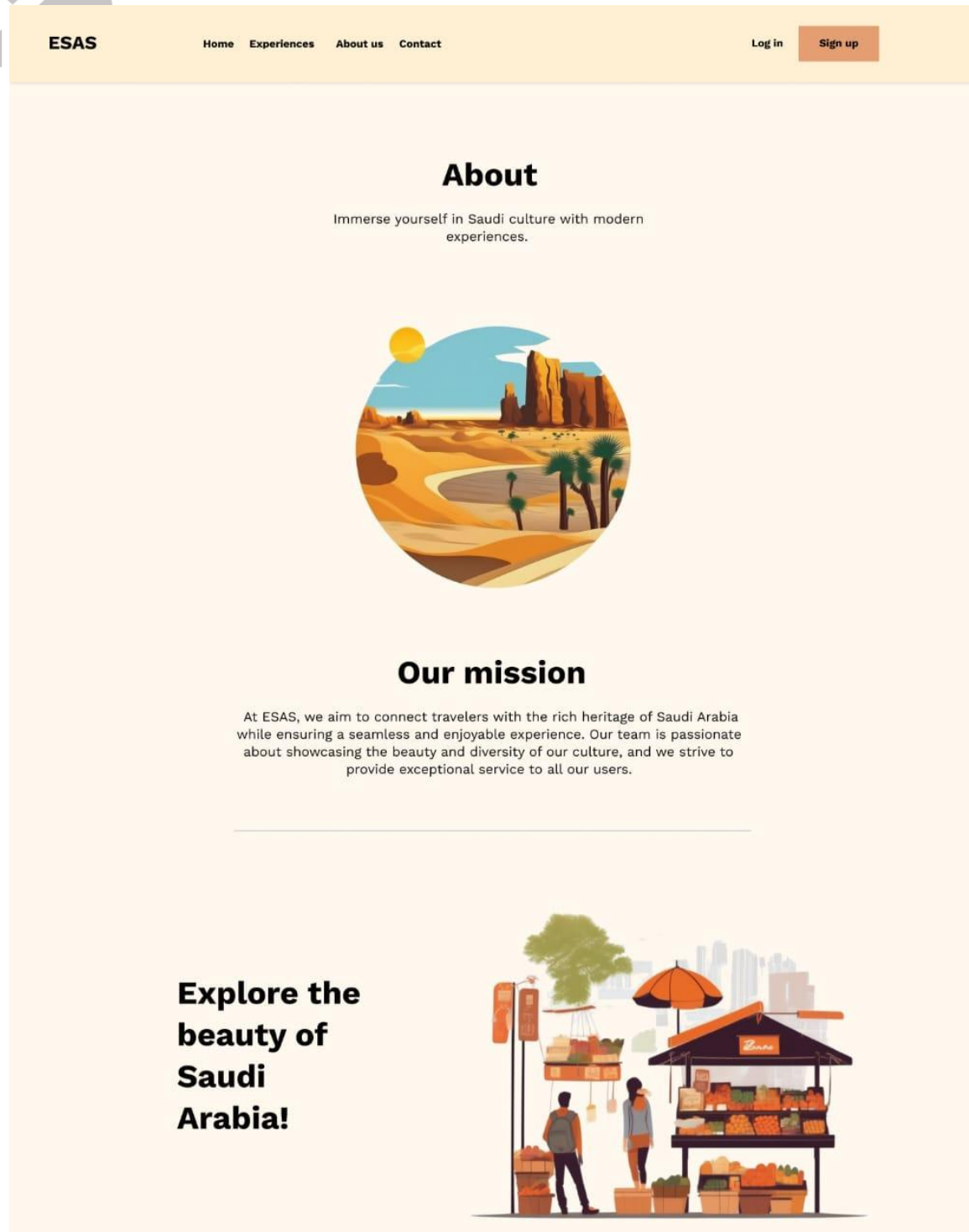


Figure 4-6: ESAS Demo About Us Page Top Half

Team

Meet the passionate individuals behind ESAS.



Ahmad Bakhsh
Software Engineer



Khaled Jameel
Software Engineer



Ayman Al-Tayaf
Software Engineer



Yahya Al-Sharif
Software Engineer



Ahmad Felemban
Software Engineer



Husam Khan
Software Engineer

Join us in experiencing

Start your adventure today.

[Get started](#)



[Terms & conditions](#)
[Help Center](#)
[Privacy](#)
[User Guide](#)
[Get in Touch](#)

Figure 4-7: ESAS Demo About Us Page Bottom Half

4.3. Structural Design Overview

This chapter presents the structural and data design of the Experience Saudi As a Saudi (ESAS).

The design is organized into three main components:

1. Identity & Access Design – defines the user hierarchy, roles, and system authentication/authorization structure.
2. Experience & Booking Design – models the main activities such as experience listings, bookings, payments, and user feedback.
3. Governance & Marketing Design – handles advertisement campaigns, impressions, and limited-time offers.

Each part includes class diagrams, followed by detailed descriptions and sequence diagrams showing the system behavior. Finally, an Entity–Relationship (ER) Diagram is provided to represent the database schema.

4.3.1. Class Diagrams

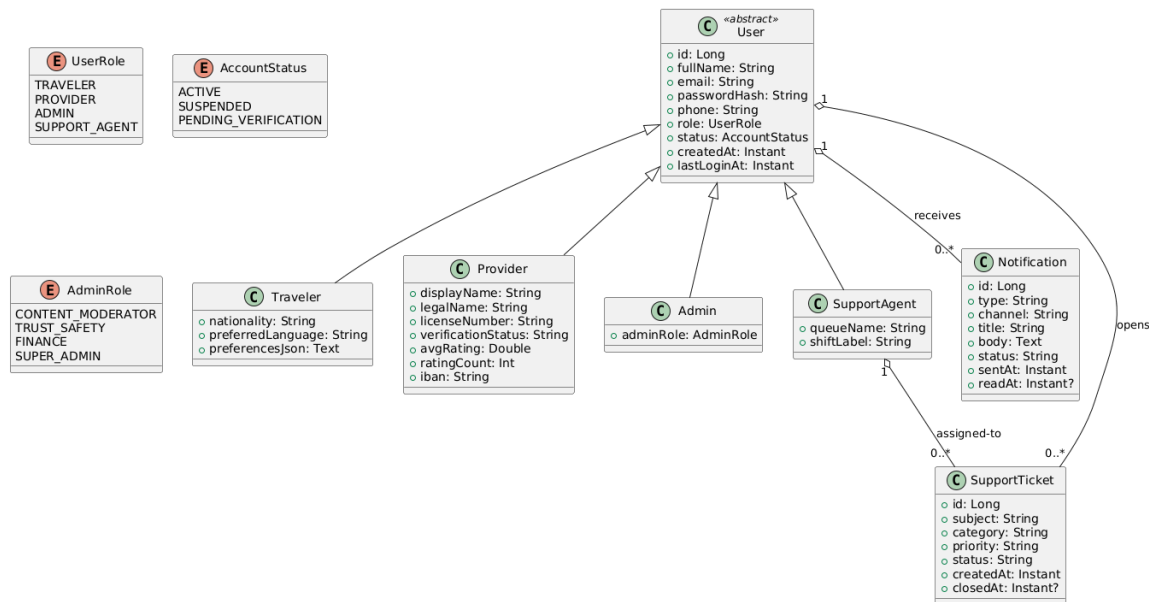


Figure 4-8: Identity & Access Class Diagram

This section models how users and system actors interact with authentication and notifications. The User class is abstract, serving as the base class for all types of users,

including Traveler, Provider, Admin, and Support Agent. Notifications and support tickets are also handled within this structure.

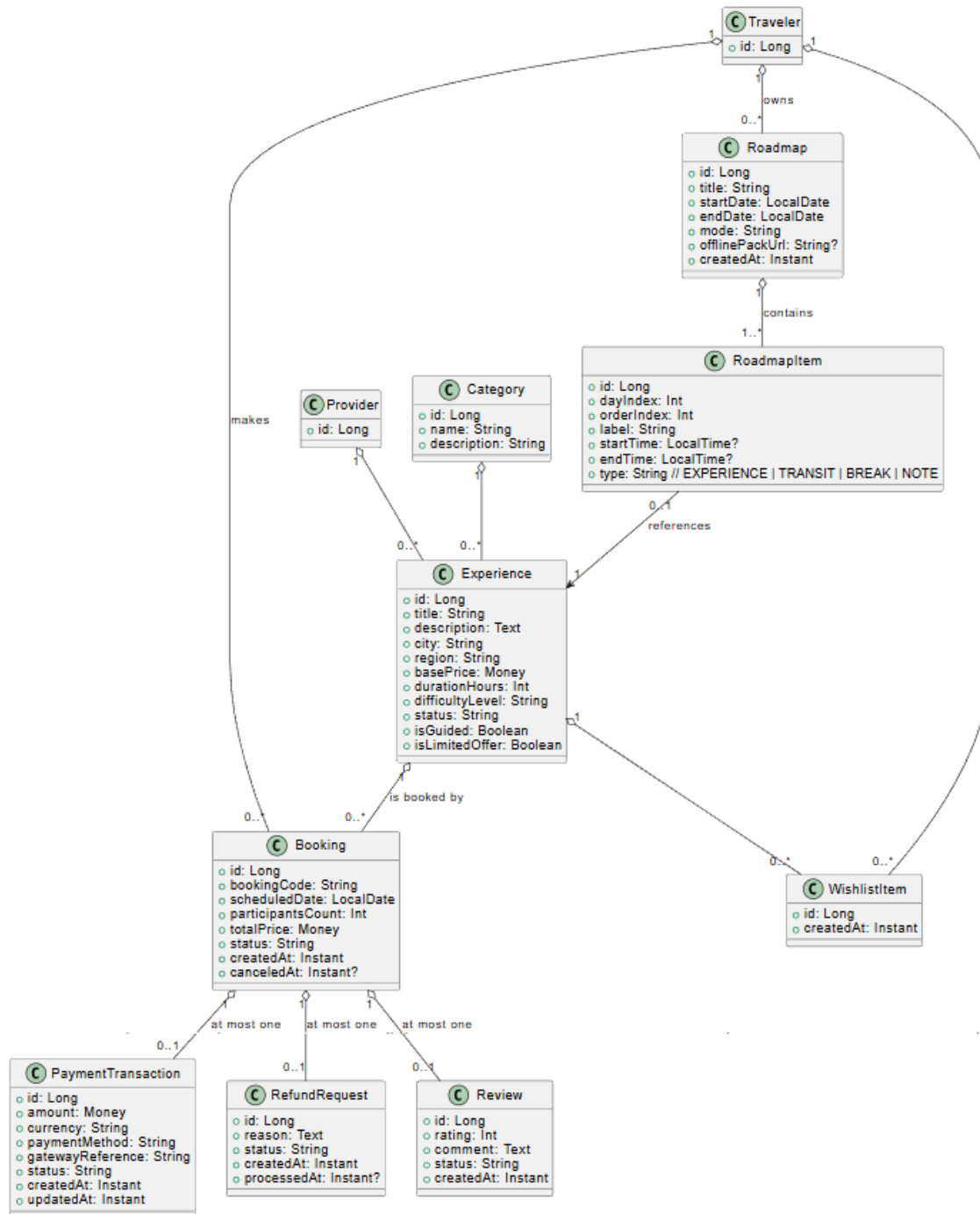


Figure 4-9: Experience & Booking Design Class Diagram

This section models the core business logic of ESAS: experiences, bookings, payments, refunds, and reviews. It also includes traveler-generated roadmaps (self-guided itineraries) and wishlist items. Each booking can have at most one payment, refund, or review, ensuring data consistency.

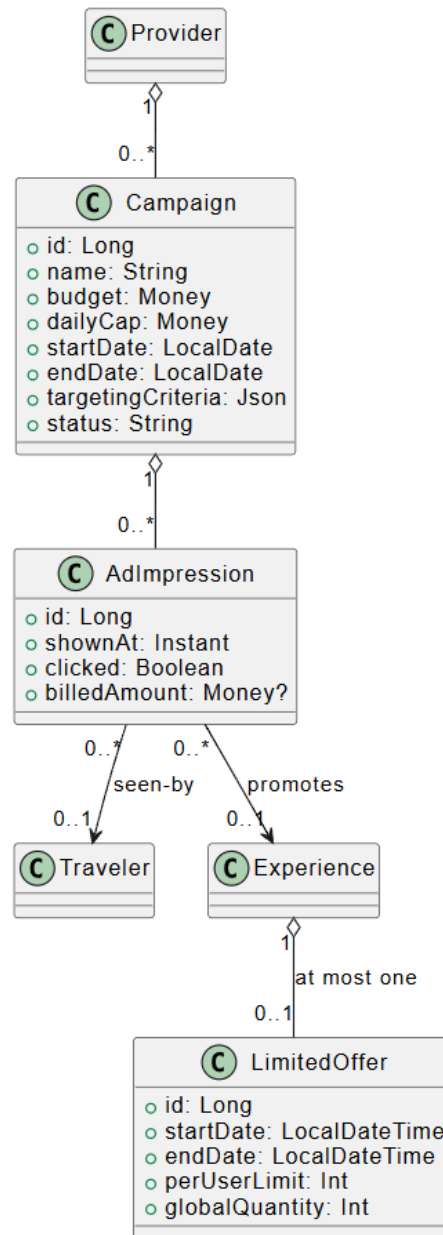


Figure 4-10: Governance & Marketing Class Diagram

This component manages marketing campaigns, advertisements, and limited offers. Providers can create campaigns to promote experiences, while the system tracks ad

impressions and connects them, optionally to travelers and experiences. Each experience may have at most one Limited Offer active at a time.

4.3.2. Class Diagram Description

The class diagrams present the overall structure of the ESAS system by combining user management, core business operations, and marketing governance. Together, they show how system actors interact with experiences, bookings, transactions, support services, and promotional components in an integrated design.

The Identity & Access Design defines a unified user model using the abstract User class, extended by Traveler, Provider, Admin, and Support Agent, enabling role-based access, notifications, and support ticket handling. The Experience & Booking Design models the main system functionality, where providers offer experiences and travelers create bookings, payments, refunds, reviews, roadmaps, and wishlists with clear one-to-one constraints for data consistency. The Governance & Marketing Design allows providers to promote experiences through campaigns and advertisements, while enforcing rules such as having at most one active limited offer per experience.

4.3.3. Sequence Diagrams

The following diagram shows the process when a traveler books a guided experience, from selection, availability checking, payment authorization, to notification delivery.

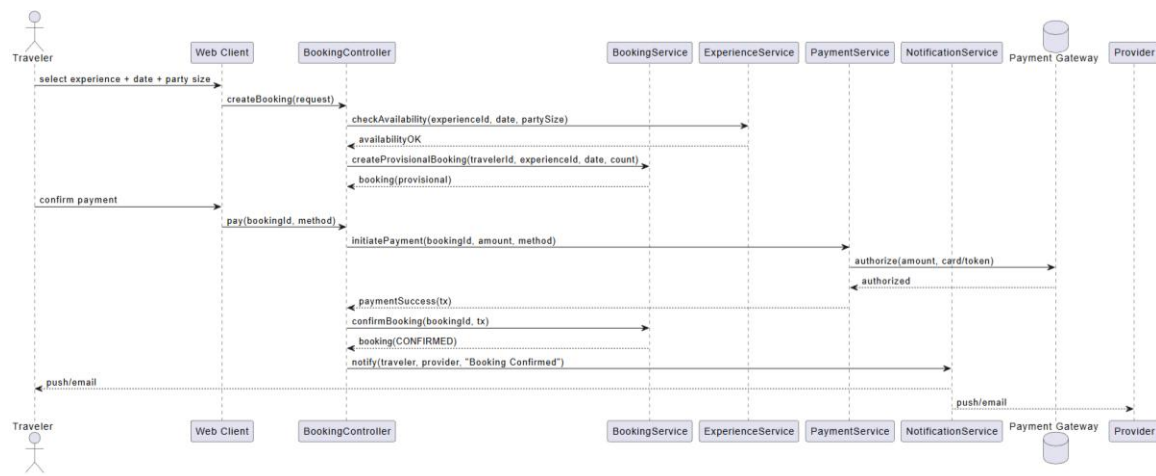


Figure 4-11: Book Guided Experience Sequence Diagram

This sequence describes how the system automatically generates a roadmap based on traveler inputs, interests, and time constraints. The optional offline pack generation is handled by the Map Service.

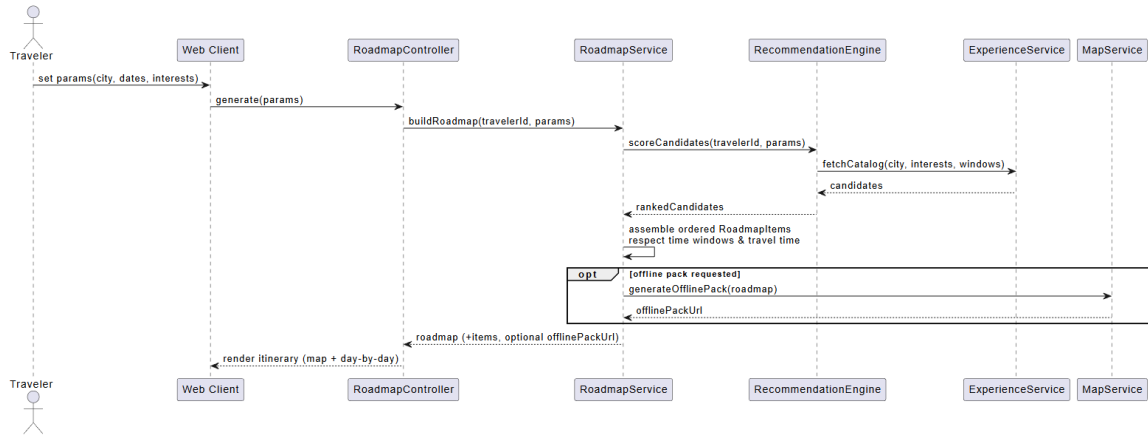


Figure 4-12: Generate Self-Guided Roadmap Sequence Diagram

This sequence shows how providers create or edit their experiences and submit them for admin review. Admins receive a notification whenever a new experience requires moderation.

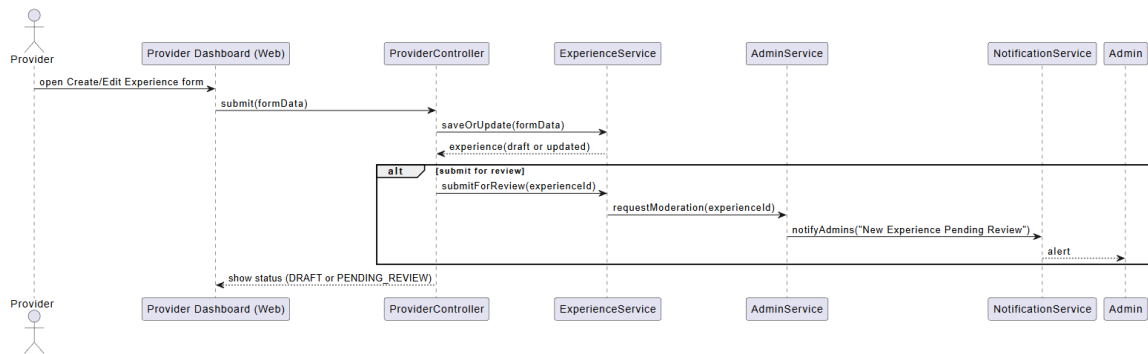


Figure 4-13: Provider Creates/Updates Experience Sequence Diagram

4.3.4. Sequence Diagram Description

The sequence diagrams illustrate the runtime behavior of the ESAS system and how its main components interact across different layers. They show coordination between clients, controllers, services, external systems, and notification mechanisms to support core system operations.

The Book Guided Experience sequence demonstrates the transactional flow of booking, payment authorization, and notification delivery. The Generate Self-Guided Roadmap sequence highlights automated itinerary generation through collaboration between roadmap, recommendation, experience, and map services. The Provider Creates/Updates Experience sequence represents the governance flow, where providers submit experiences for moderation and administrators are notified when review is required.

Together, these sequences provide a concise and integrated view of user actions, system intelligence, and administrative control within the ESAS platform.

4.3.5. ER Diagram

The ER diagram provides the logical database model for ESAS. It defines all entities, their attributes, and relationships, including one-to-one, one-to-many, and optional links. It also highlights unique constraints to maintain data integrity, such as ensuring only one limited offer per experience and one payment per booking.

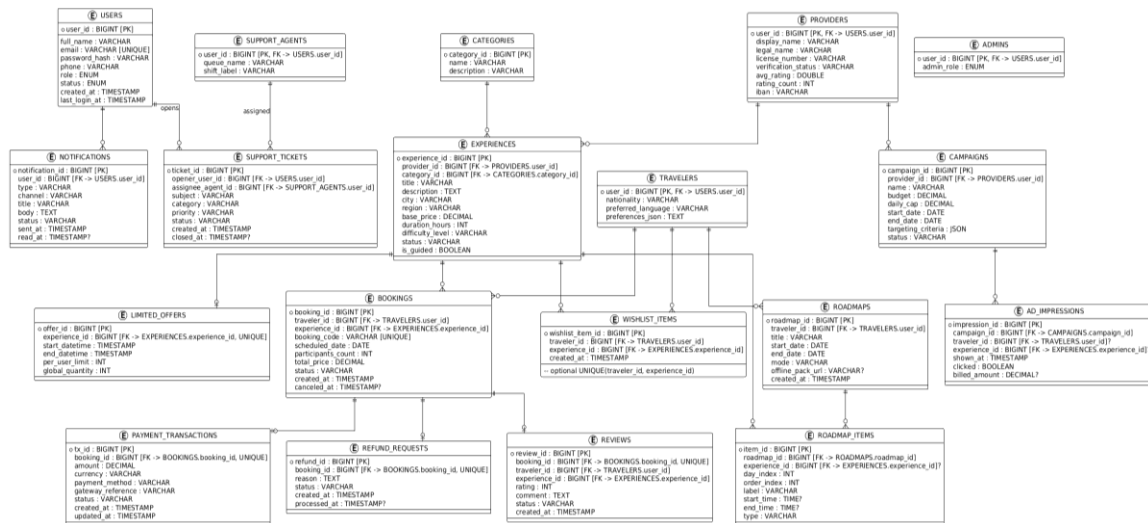


Figure 4-14: ER Diagram and Relation Schema

Implementation Notes:

- **Inheritance:** Implemented using TABLE_PER_SUBCLASS, where each user subtype references the base USERS table by foreign key.
- **Uniqueness:** Enforced by database-level constraints (e.g., UNIQUE (booking_id) in payments, refunds, and reviews).

- **Optional Relationships:** Handled with nullable foreign keys (e.g., experience_id in ROADMAP_ITEMS).
- **Scalability:** Campaigns and ad impressions are indexed for analytics and performance optimization.

Chapter 5: System Implementation & Validation

5.1. Introduction

This chapter describes the implementation of the ESAS prototype and the validation activities used to assess whether the developed system satisfies its main functional and non-functional requirements. The implementation focuses on a backend-first Saudi tourism marketplace where the backend manages authentication, users, experiences, bookings, cart operations, simulated payment flows, provider approval, administrative moderation, and system data. The frontend provides a bilingual Arabic/English interface for travelers, providers, and administrators while consuming backend REST APIs.

5.2. Implementation Environment

Component	Technology / Tool	Purpose
Backend language	Java 21	Main backend programming language.
Backend framework	Spring Boot 3.5.13, Spring Web MVC	REST API development and backend application structure.
Security	Spring Security, JWT, BCrypt	Authentication, authorization, password hashing, and role-based access control.
Database	PostgreSQL	Persistent storage for users, roles, providers, experiences, bookings, carts, wallet data, and related records.
Database migration	Flyway	Database schema versioning and seed data management.
Frontend	Flutter, Dart	Web/Android user interface implementation.
Localization	Flutter ARB files, generated localization support	Arabic and English interface text with RTL/LTR support.
API style	REST APIs, JSON request/response bodies	Communication between frontend and backend.
API documentation	OpenAPI 3.0.3 YAML	API contract and documentation reference.
Development tools	Visual Studio Code / IntelliJ IDEA, Git, GitHub	Development, version control, and collaboration.

Demo/deployment tools	Docker and Docker Compose	Local PostgreSQL/demo environment.
Testing/validation tools	Maven tests, Spring Boot integration tests, Flutter analyze, Flutter test, and manual scenario testing	Validation of backend, frontend, and user flows.

Table 5-1: Implementation Environment

The current implementation is designed as a Docker-oriented academic prototype suitable for demonstration. The backend exposes REST APIs and connects to PostgreSQL, while the Flutter frontend consumes backend APIs and presents localized Arabic and English screens. Production cloud deployment is considered future work unless confirmed separately by deployment evidence.

5.3. System Architecture Implementation

The implemented ESAS architecture follows a client-server and layered backend design. The Flutter frontend acts as the client application, while the Spring Boot backend exposes REST APIs and manages authentication, business rules, and persistence. The backend follows a controller-service-repository pattern: controllers expose endpoints, services enforce business logic, repositories communicate with the database, and DTOs define request and response contracts. This design improves maintainability and separates presentation behavior from domain and data rules.

The main architecture layers are:

- Frontend layer: Flutter UI for traveler, provider, and admin-facing screens.
- API layer: Spring Boot REST controllers.
- Security layer: Spring Security with JWT-based authentication and role-based access control.
- Business layer: Services for authentication, catalog, cart, simulated checkout, bookings, provider workflows, admin workflows, wallet/payment simulation, and validation.
- Data layer: Spring Data JPA repositories and PostgreSQL database.
- Migration layer: Flyway database migrations and seed data.
- Static media layer: backend-served static demo images for experiences.

- Documentation layer: OpenAPI contract and project documentation.
- Future integration layer: payment gateways, maps, weather services, notifications, and cloud object storage.

5.4. Backend Implementation

The backend is the core of the ESAS prototype. It manages authentication, authorization, catalog data, cart operations, simulated checkout, booking creation, wallet/payment simulation, provider onboarding, provider experience submission, administrative approval, moderation, and audit-related views. The system follows a controller-service-repository pattern, where controllers expose REST endpoints, services enforce business rules, and repositories communicate with PostgreSQL. Spring Security and JWT are used to protect authenticated endpoints and enforce role-based access for traveler, provider, and administrator functions.

The public catalog is backend-driven and is designed to show approved content. This supports the project's backend-first principle, where the frontend displays data received from backend APIs rather than acting as the source of truth for domain data.

For the current prototype, payment is represented through simulated checkout and simulated payment confirmation flows. The system can create bookings, update payment status, provide demo receipts, and clear the traveler's cart after successful simulated checkout. Real integration with certified local payment gateways such as Mada, Apple Pay, STC Pay, Moyasar, HyperPay, or Tap is future work and must not be described as fully implemented.

The backend also includes provider and admin functionality. Providers can move through onboarding/verification-related flows and submit/manage their own experiences according to the implemented scope. Administrators can review provider approvals, moderate submitted experiences, view operational information, and access audit/report-related functionality according to the current prototype.

5.5. Database Implementation

The ESAS database is implemented using PostgreSQL. Flyway migrations are used to manage schema evolution and seed initial demonstration data. The database includes records for users, roles, providers, destinations, experiences, guided packages, bookings, payment transactions, wallet data, wishlists, carts, cart items, audit logs, and other schema elements. Some tables support future features documented in the broader system design, but only runtime-confirmed features should be described as implemented.

The implemented prototype uses seeded demonstration data and static experience image paths served by the backend. The ER diagram in Chapter 4 represents the system data model and should be referenced rather than recreated. If any table exists only in migrations but has no confirmed controller/service/UI workflow, it should be described as schema support or future scope, not as fully implemented.

5.6. Frontend Implementation

The frontend is implemented using Flutter and Dart. It provides a bilingual Arabic/English interface and communicates with the backend through REST APIs. The frontend is responsible for layout, navigation, localization, responsive behavior, and user interaction, while business-critical data and workflow rules remain controlled by the backend.

The current frontend prototype includes backend-driven home/catalog pages, experience details, guided packages, cart and checkout pages, booking/My Trips page, wallet page, account/profile page with partial profile functionality, admin pages for stats/users/bookings/provider approvals/experience moderation/audit, and provider pages for dashboard summary, experiences, experience editor/submission, and bookings. The contact page currently exists as a frontend-only form unless backend support is confirmed. Loading, empty, error, and retry states are used broadly in backend-connected screens.

5.7. Implemented Features

Feature Area	Current Prototype Status	Notes
Health endpoint	Implemented	Backend health endpoint and actuator health configuration exist.
Authentication and role handling	Implemented	Signup, login, refresh, logout, JWT, BCrypt, and RBAC are implemented.
User profile / me profile	Partially implemented	Basic /me and profile update exist, but full profile fields are limited.
Public experience catalog	Implemented	Backend-driven approved catalog browsing.
Experience filtering/search/sorting	Partially implemented	Keywords, city/region/category/price/sort are implemented; not every planned filter is complete.
Experience details	Implemented	Approved-only experience detail retrieval is implemented.
Guided packages	Partially implemented	Public read/list/detail paths exist; full provider package management is not confirmed.
Traveler cart	Implemented	An authenticated traveler cart is backend-mediated.
Simulated checkout	Implemented	Creates bookings and clears cart; not real payment gateway processing.
Booking management	Implemented/Partially implemented	Booking creation, history, cancellation/payment status/demo receipt paths exist; the full commercial lifecycle is not complete.
Wishlist	Implemented	Add/remove/list wishlist functionality exists.
Wallet/payment simulation	Implemented/Partially implemented	Wallet reads and simulated/wallet payment flows exist; real settlement is not implemented.
Provider onboarding/approval	Implemented/Partially implemented	Provider signup, verification submission, and admin approval flow exist.
Provider offering management	Implemented/Partially implemented	Providers can create/update/submit experiences; full advanced provider operations are future work.

Admin provider approval	Implemented	Admin provider queue/detail/decision with audit support exists.
Admin experience moderation	Implemented	Admin experience queue and approval/rejection flow exist.
Admin reports/audit	Implemented/Partially implemented	Audit/report-related functionality exists; full analytics/dispute dashboard is future work.
Reviews and ratings	Documented/schema only / Future work	Tables/OpenAPI references exist, but no runtime controller/service was confirmed.
Notifications	Documented/schema/log-only partial / Future work	No production notification API/UI confirmed.
Messaging/support	Documented/schema only / Future work	No runtime controller/service confirmed.
Roadmaps/offline packs	Documented/schema/OpenAPI only / Future work	No runtime implementation confirmed.
AI/weather recommendations	Future work	Not confirmed in runtime code.
Map/routing	Future work	OpenAPI may document map endpoints, but no runtime implementation has been confirmed.
Provider payouts/campaigns/ads	Future work	Schema/OpenAPI may exist, but no runtime workflow has been confirmed.
Bilingual UI	Implemented	Arabic/English localization and RTL-aware UI patterns exist.
Static demo images	Implemented	Backend serves demo experience images.
Production cloud deployment	Future work	A local Docker/PostgreSQL setup and deployment checklist exists, but production deployment was not confirmed.

Table 5-2: Implemented Features Summary

5.8. Validation and Testing

The ESAS prototype was prepared for validation using a combination of automated checks and manual scenario testing. Automated validation focuses on backend integration tests, backend compilation, frontend static analysis, frontend tests, and frontend build checks. Manual validation focuses on user journeys such as authentication, browsing experiences,

viewing details, adding items to the cart, performing simulated checkout, checking bookings, switching languages, and verifying role-based access behavior.

The repository includes backend integration tests and Flutter widget tests. These tests cover authentication, catalog filtering, booking, payment simulation, profile/wishlist, admin approval, and contact-page UI behavior. However, the latest test execution result must be confirmed by running the documented validation commands before final submission. Therefore, validation status entries should be written as “To be confirmed” unless test output is provided.

5.8.1. Backend Validation

Backend validation should be confirmed by running the Maven compilation and test commands. The repository includes integration tests for authentication, catalog, booking, payment simulation, profile/wishlist, and admin workflows. Current pass/fail status must remain “To be confirmed” unless the commands are executed and results are provided.

5.8.2. Frontend Validation

Frontend validation should be confirmed by running Flutter analysis, tests, and web build commands. Existing widget tests cover selected UI behavior, but full validation requires running the latest commands before final submission.

5.8.3. Manual Test Scenarios

Test ID	Scenario	Expected Result	Status
T-01	Start backend and database	Backend connects to PostgreSQL, and the health endpoint is reachable	To be confirmed
T-02	Open the frontend application	Application loads and displays the correct initial screen	To be confirmed
T-03	Register or log in as a traveler	Traveler session is created, and role-based access is applied	To be confirmed
T-04	Browse public experiences	Approved experiences are loaded from the backend	To be confirmed
T-05	Filter/search experiences	Results update according to supported backend filters	To be confirmed

T-06	View experience details	Selected approved experience details are displayed	To be confirmed
T-07	Add an eligible item to the cart	Item is stored through the backend cart API	To be confirmed
T-08	View and update cart	Cart contents are retrieved and updated through backend APIs	To be confirmed
T-09	Perform simulated checkout	Booking is created, and the cart is cleared	To be confirmed
T-10	View booking history	Traveler bookings are displayed from the backend data	To be confirmed
T-11	Cancel or inspect booking/payment status	System updates or displays booking/payment status according to the implemented workflow	To be confirmed
T-12	Add/remove wishlist item	Wishlist updates through backend APIs	To be confirmed
T-13	Switch language Arabic/English	UI text and direction update correctly	To be confirmed
T-14	Access restricted traveler/provider/admin pages	Unauthorized users are blocked or redirected	To be confirmed
T-15	Provider submits experience	Provider-owned experience is created/submitted according to the implemented provider flow	To be confirmed
T-16	Admin reviews the provider request	Admin can approve/reject the provider request according to the implemented flow	To be confirmed
T-17	Admin moderates submitted experience	Admin can approve/reject submitted experience according to the implemented flow	To be confirmed
T-18	Frontend handles empty/error states	User receives safe empty/error/retry messages instead of raw backend exceptions	To be confirmed

Table 5-3: Manual Test Scenarios

5.9. Validation Results

The available repository evidence shows that ESAS includes implementation and test support for the main academic demonstration goals: backend-driven catalog browsing, authentication and role-aware workflows, cart and simulated checkout, booking flows, provider submission/approval direction, admin moderation/audit, bilingual frontend

interaction, and maintainable backend structure. Final validation results must be confirmed by running the documented backend and frontend commands before submission.

Advanced features remain future work, including real payment gateway integration, production cloud deployment, AI/weather-based recommendations, map/routing, offline roadmap packs, review runtime, notifications, messaging/support, refunds/disputes, provider payouts, and advertising campaign billing.

5.10. Implementation Limitations

- Real payment gateway integration is not implemented.
- Payment is simulated for the prototype.
- AI/weather recommendation is not implemented in the runtime code.
- Map/routing functionality is not confirmed as implemented.
- Smart roadmap generation and offline roadmap downloads are not implemented in the runtime code.
- Reviews/ratings runtime API/UI is not confirmed.
- Notifications delivery/API/UI is not confirmed.
- Messaging/support ticket runtime is not confirmed.
- Refund/dispute runtime workflow is not confirmed.
- Full provider payouts, campaigns, ads, and limited offers runtime are future work.
- Full editable traveler profile beyond basic name/email is limited.
- Full provider dashboard for payouts/campaigns/advanced availability/pricing is future work.
- Full admin dashboard for disputes/refunds/rich analytics/charts is future work.
- Production cloud deployment is not confirmed.
- Security hardening, performance testing, and production monitoring need further work before public deployment.



Chapter 6: Conclusion & Future Work

6.1. Conclusion

ESAS demonstrates a backend-first tourism marketplace prototype focused on helping travelers discover and book authentic Saudi experiences. The project addresses limitations in generic tourism platforms by proposing Saudi-curated content, guided packages, self-guided roadmap concepts, provider verification, and backend-managed booking workflows. The implemented prototype establishes the core foundation through a Spring Boot backend, PostgreSQL database, Flyway migrations, Flutter frontend, JWT-based security, and backend-driven catalog, cart, simulated checkout, booking, provider, and admin workflows.

The current prototype is suitable for academic demonstration because it proves the feasibility of the main system architecture and core traveler journey while also showing provider and admin workflow foundations. It provides a strong basis for future expansion into real payment processing, recommendation services, map and roadmap features, mobile deployment, notifications, and production-grade operations.

6.2. Achievements

The following achievements were accomplished through the ESAS project:

- Designed the ESAS platform scope and core requirements.
- Developed backend architecture using Java 21 and Spring Boot 3.5.13.
- Implemented authentication, JWT security, BCrypt password hashing, and role-based access control.
- Used PostgreSQL and Flyway for persistence, migrations, and seed data.
- Built a Flutter frontend with Arabic/English localization and RTL-aware UI patterns.
- Implemented backend-driven public catalog browsing for approved experiences.
- Implemented experience details and partially guided package browsing.
- Implemented traveler cart and simulated checkout flow.
- Implemented booking creation/history and related booking/payment status paths according to prototype scope.

- Implemented wishlist functionality.
- Implemented wallet read/payment simulation functionality according to the prototype scope.
- Implemented provider onboarding/submission foundations.
- Implemented admin provider approval, experience moderation, and audit/report-related foundations.
- Served static demo experience images from the backend.
- Prepared diagrams and design models, including BPMN, activity, use case, class, sequence, and ER diagrams.
- Documented system requirements and design.
- Prepared automated and manual validation approach through backend integration tests, Flutter tests, and scenario testing.

6.3. Limitations

The following limitations apply to the current ESAS prototype:

- Current payment flow is simulated and not connected to real local payment gateways.
- AI/weather-based recommendations are planned but not implemented in runtime code.
- Smart map/routing and self-guided roadmap generation are planned/future features.
- Offline roadmap downloads are planned but not implemented in the runtime code.
- Reviews/ratings runtime API/UI is not confirmed.
- Notifications and messaging/support runtime workflows are future work.
- Refund/dispute workflows are not fully implemented.
- Provider payouts, advertising campaigns, limited offers runtime, and billing workflows are future work.
- Some provider and admin pages are partial prototypes rather than complete operational dashboards.
- Full editable traveler profile fields are limited.
- Production-grade cloud deployment, monitoring, performance testing, and security hardening are not fully finalized.

6.4. Future Work

The following areas are identified for future development:

- Integrate certified local payment gateways such as Mada, Apple Pay, and STC Pay through providers like Moyasar, HyperPay, or Tap.
- Add real AI/season/weather recommendation logic.
- Add interactive map exploration and smart route planning.
- Implement self-guided roadmap generation.
- Implement downloadable offline roadmaps.
- Complete reviews and ratings of the runtime API/UI.
- Improve notification support through email, SMS, or push notifications.
- Implement traveler-provider messaging and support tickets.
- Complete refund, dispute, and cancellation policy workflows.
- Complete provider onboarding and verification workflows.
- Complete provider payout and finance operations.
- Complete provider campaign, advertisement, and limited-offer tools.
- Complete admin moderation, dispute handling, refund oversight, audit reporting, and analytics.
- Expand mobile testing on Android and iOS.
- Deploy the system to a production cloud environment.
- Add stronger security testing, performance testing, usability testing, and monitoring.

6.5. Final Statement

Overall, ESAS provides a strong foundation for a Saudi-focused tourism platform that supports Vision 2030 by improving access to authentic local experiences and supporting Saudi guides and small tourism providers. The implemented prototype demonstrates the technical feasibility of the core architecture and main traveler, provider, and administrator workflows. With continued development, the prototype can evolve into a scalable, secure, and user-friendly platform for travelers and local tourism stakeholders.

References / Bibliography

The following references were consulted during the development of this report. References are listed in citation order:

- [1] Saudi Vision 2030. (2024). *Saudi Vision 2030*. <https://www.vision2030.gov.sa/en>
- [2] VMware Tanzu. (2024). *Spring Boot reference documentation*. Spring.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [3] Spring Security Team. (2024). *Spring Security reference*. Spring.
<https://docs.spring.io/spring-security/reference/>
- [4] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*.
<https://www.postgresql.org/docs/>
- [5] Redgate. (2024). *Flyway documentation*. <https://documentation.red-gate.com/fd>
- [6] Google. (2024). *Flutter documentation*. <https://docs.flutter.dev/>
- [7] Docker Inc. (2024). *Docker documentation*. <https://docs.docker.com/>
- [8] OpenAPI Initiative. (2020). *OpenAPI specification version 3.0.3*.
<https://spec.openapis.org/oas/v3.0.3>
- [9] IEEE Computer Society. (1998). *IEEE recommended practice for software requirements specifications (IEEE Std 830-1998)*. IEEE.
<https://standards.ieee.org/standard/830-1998.html>
- [10] National Cybersecurity Authority. (2018). *Essential cybersecurity controls (ECC-1:2018)*. Kingdom of Saudi Arabia. <https://nca.gov.sa/ecc-en.pdf>
- [11] Saudi Data & AI Authority. (2021). *Personal Data Protection Law*. Kingdom of Saudi Arabia. <https://sdaia.gov.sa/en/Research/Pages/DataProtection.aspx>
- [12] Moyasar. (2024). *Moyasar payment gateway documentation*.
<https://moyasar.com/docs>

[13] HyperPay. (2024). *HyperPay integration documentation*.
<https://developer.hyperpay.com>

[14] Auth0. (2024). *JSON web token (JWT) best practices*.
<https://auth0.com/docs/secure/tokens/json-web-tokens>

[15] Umm Al-Qura University, College of Computer and Information Systems, Software Engineering Department. (2025). *Graduation project template: GP-Template-UQU-CIS-SE*.